AI-POWERED COGNITIVE SCREENING & DEMENTIA RISK ANALYSIS PLATFORM

Deployment Report

Project ID: 25-26J-439

Senaratne H D	IT22358998
Dissanayake M.P.S.	IT22901644
Piruthivi K	IT22358820
Natheer H A H	IT22888952

BSc (Hons) in Information Technology Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

April 2026

AI-POWERED COGNITIVE SCREENING & DEMENTIA RISK ANALYSIS PLATFORM

Deployment Report

Project ID: 25-26J-439

Senaratne H D	IT22358998
Dissanayake M.P.S.	IT22901644
Piruthivi K	IT22358820
Natheer H A H	IT22888952

BSc (Hons) in Information Technology Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology Sri Lanka

April 2026

Table of Contents

1. Smart Health Monitoring and Dementia Risk Analysis Deployment 9
   1.1. Executive Summary 9
      1.1.1. Overview 9
      1.1.2. Key Metrics 9
   1.2. Deployment Objectives 10
      1.2.1. Goals 10
      1.2.2. Scope 10
   1.1. Pre-Deployment Activities 11
      1.1.1. Environment Preparation 11
      1.1.2. Testing 11
      1.1.3. Backup Plan 13
   1.2. Deployment Details 13
      1.2.1. Date and Time 13
      1.2.2. Team Members 13
      1.2.3. Steps and Procedures 13
   1.3. Tools and Technologies 17
   1.4. Deployment Checklist 17
      1.4.1. Pre-Deployment Checklist 17
      1.4.2. Deployment Checklist 17
      1.4.3. Post-Deployment Checklist 18
   1.5. Issues and Resolutions 18
      1.5.1. Encountered Issues 18
      1.5.2. Solutions 18
      1.5.3. Impact Analysis 19
   1.6. Performance and Monitoring 19
      1.6.1. Performance Metrics 19
      1.6.2. Monitoring Results 19
      1.6.3. User Feedback 19
   1.7. Post-Deployment Activities 20
      1.7.1. Validation and Testing 20
      1.7.2. Documentation Updates 20

1.7.3. Recommendations 21
1.8. Lessons Learned 21
1.8.1. Successes 21
1.8.2. Areas for Improvement 22
1.8.3. Recommendations 22
2. Handwriting-Based Dementia Screening Module Deployment 22
2.1 Executive Summary 22
2.1.1. Overview 22
2.1.2. Key Metrics 23
2.2. Deployment Objectives 23
2.2.1. Goals 23
2.2.2. Scope 24
2.3. Pre-Deployment Activities 25
2.3.1. Environment Preparation 25
2.4 Testing 26
2.11.2 Unit Testing 26
Unit testing was conducted to validate the correctness and reliability of individual components within the handwriting-based dementia screening module. Each function was tested in isolation to ensure that it performs as expected under controlled conditions. 26



..... 27

2.11.2 Integration Testing 27

2.3.2. Backup Plan 29

2.5. Deployment Details 29

2.5.1. Date and Time 29

2.5.2. Steps and Procedures 29

2.6. Tools and Technologies 37

2.7. Deployment Checklist 38

2.7.1. Pre-Deployment Checklist 38
2.7.2. Deployment Checklist 38
2.7.3. Post-Deployment Checklist 38
2.8. Issues and Resolutions 39
2.8.1. Encountered Issues 39
2.8.2. Solutions 40
2.8.3. Impact Analysis 40
2.9. Performance and Monitoring 41
2.9.1. Performance Metrics 41
2.9.2. Monitoring Results 41
2.9.3. User Feedback 41
2.10. Post-Deployment Activities 43
2.10.1. Validation and Testing 43
2.10.2. Documentation Updates 43
1.3. Recommendations 43
1.4. Lessons Learned 44
2.10.3. Successes 44
2.10.4. Areas for Improvement 44
2.10.5. Recommendations 45
3. Speech-Task Dementia Screening Module Deployment 45
3.1. Executive Summary 45
3.1.1. Overview 45
3.1.2. Key Metrics 45
3.2. Deployment Objectives 46
3.2.1. Goals 46
3.2.2. Scope 46
3.3. Pre-Deployment Activities 46
3.3.1. Environment Preparation 46
3.3.2. Testing 46
3.3.3. Backup Plan 47
3.4. Deployment Details 47
3.4.1. Date and Time 47

3.4.2. Team Members 47
3.4.3. Steps and Procedures 47
3.5. Tools and Technologies 47
3.6. Deployment Checklist 47
3.6.1. Pre-Deployment Checklist 47
3.6.2. Deployment Checklist 47
3.6.3. Post-Deployment Checklist 48
3.7. Issues and Resolutions 48
3.7.1. Encountered Issues 48
3.7.2. Solutions 48
3.7.3. Impact Analysis 48
3.8. Performance and Monitoring 48
3.8.1. Performance Metrics 48
3.8.2. Monitoring Results 48
3.8.3. User Feedback 48
3.9. Post-Deployment Activities 49
3.9.1. Validation and Testing 49
3.9.2. Documentation Updates 49
3.9.3. Recommendations 49
3.10. Lessons Learned 49
3.10.1. Successes 49
3.10.2. Areas for Improvement 49
3.10.3. Recommendations 49
4. MRI-Based Dementia Stage Classification Module Deployment 50
4.1. Executive Summary 50
4.1.1. Overview 50
4.1.2. Key Metrics 50
4.2. Deployment Objectives 51
4.2.1. Goals 51
4.2.2. Scope 52
4.3. Pre-Deployment Activities 53
4.3.1. Environment Preparation 53

4.3.2. Testing 55
4.3.3. Backup Plan 59
4.4. Deployment Details 59
4.4.1. Date and Time 59
4.4.2. Team Members 59
4.4.3. Steps and Procedures 59
4.5. Tools and Technologies 63
4.6. Deployment Checklist 64
4.6.1. Pre-Deployment Checklist 64
4.6.2. Deployment Checklist 64
4.6.3. Post-Deployment Checklist 64
4.7. Issues and Resolutions 64
4.7.1. Encountered Issues 64
4.7.2. Solutions 65
4.7.3. Impact Analysis 65
4.8. Performance and Monitoring 65
4.8.1. Performance Metrics 65
4.8.2. Monitoring Results 65
4.8.3. User Feedback 66
4.9. Post-Deployment Activities 66
4.9.1. Validation and Testing 66
4.9.2. Documentation Updates 66
4.9.3. Recommendations 66
4.10. Lessons Learned 67
4.10.1. Successes 67
4.10.2. Areas for Improvement 67
4.10.3. Recommendations 67

1. Smart Health Monitoring and Dementia Risk Analysis Deployment

1.1. Executive Summary

1.1.1. Overview

The Dementia Risk Prediction using smart watch health data, ML Service of the CogniCare system was successfully deployed as a cloud-based microservice to provide real-time dementia risk analysis using structured health and lifestyle data. The service is designed to process user-provided inputs such as age, medical conditions, lifestyle factors, and physiological indicators, and generate predictive outputs indicating the probability of dementia risk along with a corresponding risk level classification.

The deployment was implemented using a containerized architecture, where the backend service developed using FastAPI (Python) was packaged into a Docker container and deployed through Microsoft Azure Container Apps. This approach ensured platform independence, scalability, and ease of deployment across different environments. The system exposes RESTful API endpoints, enabling external systems or interfaces to interact with the model seamlessly.

The deployed service is publicly accessible through a secure HTTP endpoint and supports real-time interaction, allowing users or systems to submit input data and receive predictions instantly. The API utilizes structured JSON-based request and response formats, ensuring efficient and consistent communication between the client and the backend service.

The prediction endpoint processes input features such as demographic data, medical history, and health indicators, and returns a probability score along with a categorized risk level (e.g., Low, Medium, High). This structured output enables easy interpretation and integration into other applications or decision-making systems.

The deployment was optimized for low-latency inference, ensuring quick response times suitable for real-time usage scenarios. By leveraging Azure's managed infrastructure, the system benefits from automatic scaling, high availability, and reduced operational overhead.

Overall, the deployed ML service represents a robust, scalable, and efficient solution for dementia risk prediction, effectively integrating machine learning capabilities into a cloud-native architecture for real-world application.

1.1.2. Key Metrics

Metric	**Value / Status**
Deployment Platform	Azure Container Apps
Backend Framework	FastAPI (Python)
Containerization	Docker

Service Port	8000
API Endpoint	POST /predict
Average Response Time	~100–300 ms
Model Type	Machine Learning (Tabular Data)
Output	Probability + Risk Level
Resource Usage	CPU-based inference

1.2. Deployment Objectives

1.2.1. Goals

The primary goals of deploying the ML service were:

- To provide a scalable and reliable API service for dementia risk prediction using health-related input data
- To enable real-time processing of input data, allowing users to receive immediate predictions with minimal latency
- To ensure portability and consistency across environments using containerization with Docker
- To deploy the service on a cloud platform for high availability and scalability
- To maintain a simple and structured API design for easy integration with other system components
- To ensure efficient resource utilization using CPU-based inference without requiring specialized hardware
- To support future integration with frontend applications or mobile systems
- To establish a stable deployment pipeline for further system enhancements

1.2.2. Scope

- back end API service responsible for processing input data and generating dementia risk predictions
- Containerization of the application using Docker to ensure consistent runtime environments
- Deployment of the containerized service using Azure Container Apps
- Exposure of a public API endpoint for testing and integration purposes
- Support for real-time prediction requests using structured JSON inputs
- Implementation of a CPU-based inference pipeline suitable for cloud deployment
- Basic monitoring and logging through Azure services
- Modular architecture allowing independent updates and scaling of the ML service
- The scope is limited to deployment and operation of the ML prediction service and does not include frontend development or model retraining

1.1. Pre-Deployment Activities

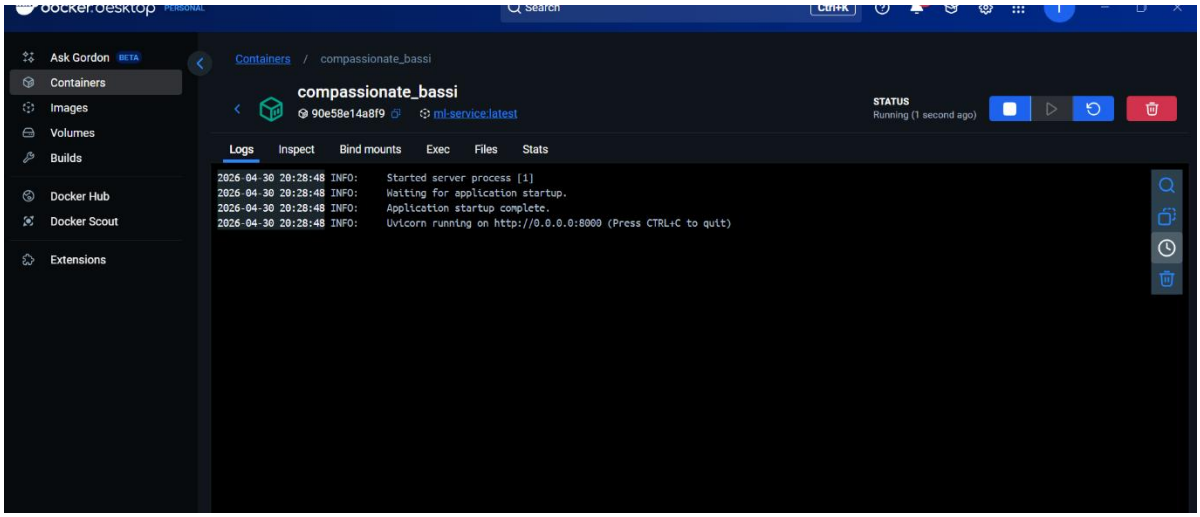
1.1.1. Environment Preparation

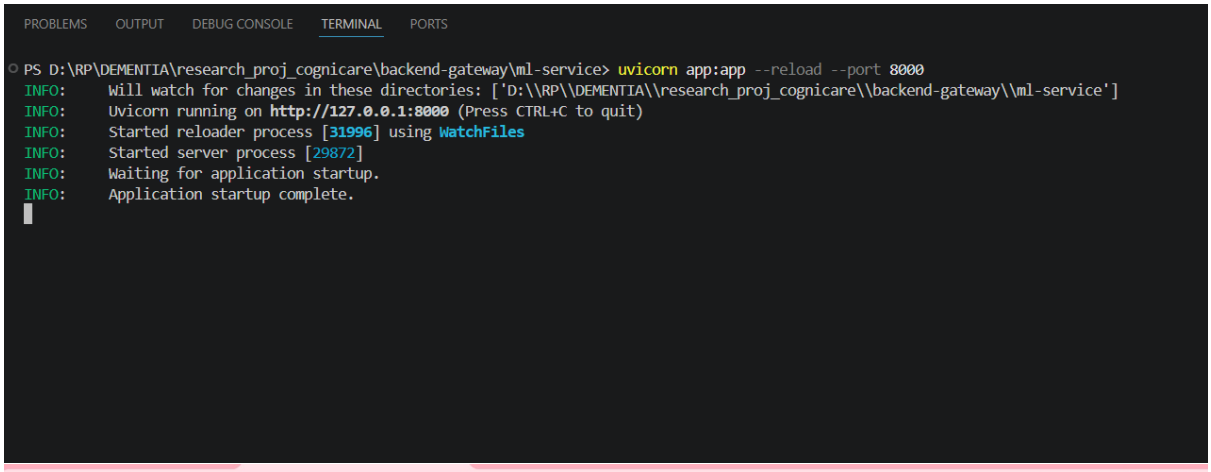
Prior to deployment, the development environment was configured to ensure compatibility with the containerized and cloud-based infrastructure. Key activities included:

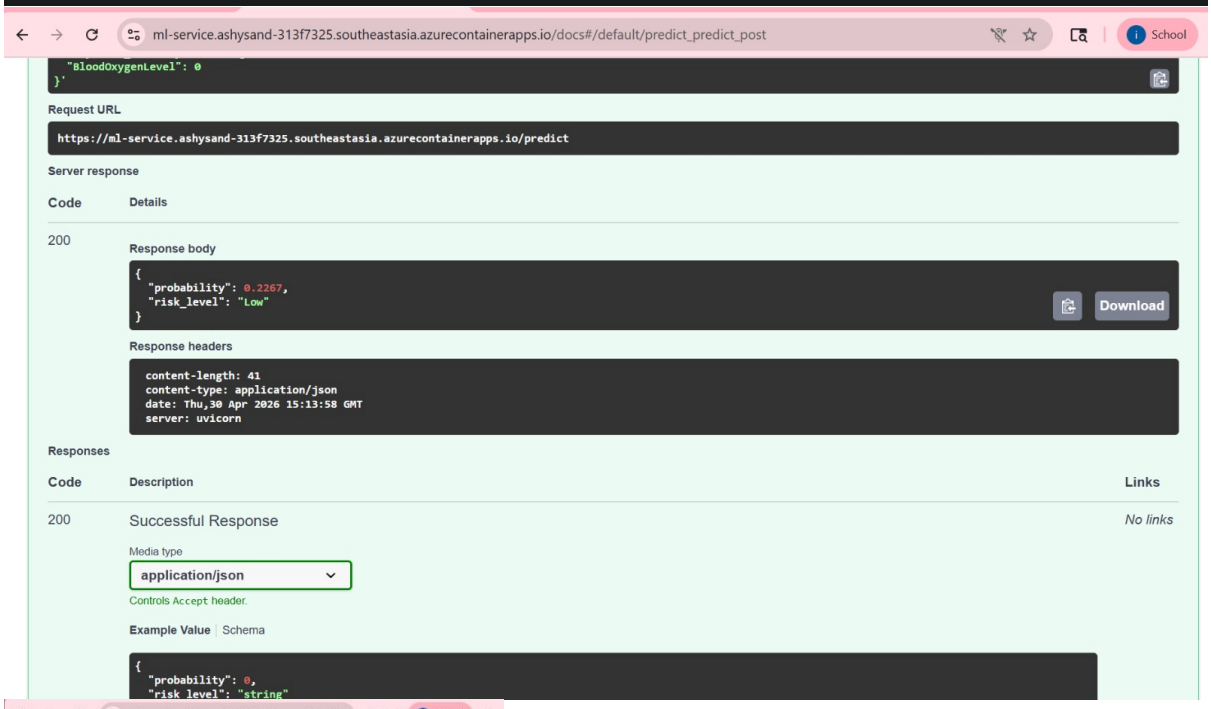
- Creating a Dockerfile to package the FastAPI application and its dependencies
- Defining required Python libraries (e.g., FastAPI, NumPy, Pandas, Scikit-learn) in the requirements.txt file
- Ensuring the application runs on a consistent port (**8000**) across local and cloud environments
- Configuring application startup using Uvicorn server
- Setting up Docker Desktop for local container testing
- Building and testing the Docker image locally before deployment
- Preparing the Azure environment, including resource group creation and container app configuration

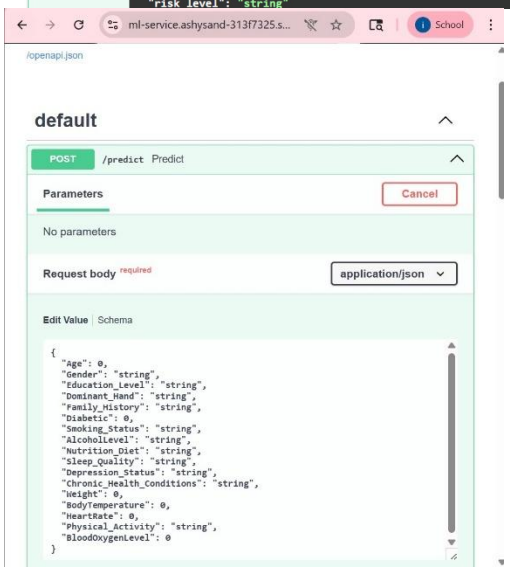
1.1.2. Testing

Local testing using Uvicorn server
Docker container testing using port mapping
API testing via Swagger UI (/docs)




PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS D:\RP\DEMENTIA\research_proj_cognicare\backend-gateway\ml-service> uvicorn app:app --reload --port 8000
INFO: Will watch for changes in these directories: ['D:\\RP\\DEMENTIA\\research_proj_cognicare\\backend-gateway\\ml-service']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [31996] using WatchFiles
INFO: Started server process [29872]
INFO: Waiting for application startup.
INFO: Application startup complete.


ml-service.ashysand-313f7325.southeastasia.azurecontainerapps.io/docs#/default/predict_predict_post
School
"BloodOxygenLevel": 0
}'
Request URL
https://ml-service.ashysand-313f7325.southeastasia.azurecontainerapps.io/predict
Server response
Code
Details
200
Response body
{
"probability": 0.2267,
"risk_level": "Low"
}
Download
Response headers
content-length: 41
content-type: application/json
date: Thu,30 Apr 2026 15:13:58 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value
Schema
{
"probability": 0,
"risk_level": "string"


ml-service.ashysand-313f7325.s...
School
/openapi.json
default
POST
/predict
Predict
Parameters
Cancel
No parameters
Request body
required
application/json
Edit Value
Schema
{
"Age": 0,
"Gender": "string",
"Education_Level": "string",
"Dominant_Hand": "string",
"Family_History": "string",
"Diabetic": 0,
"Smoking_Status": "string",
"AlcoholLevel": "string",
"Nutrition_Diet": "string",
"Sleep_Quality": "string",
"Depression_Status": "string",
"Chronic_Health_Conditions": "string",
"Height": 0,
"BodyTemperature": 0,
"HeartRate": 0,
"Physical_Activity": "string",
"BloodOxygenLevel": 0
}

1.1.3. Backup Plan

Code stored in Git repository
Docker image stored in Docker Hub
Versioning strategy planned

1.2. Deployment Details

1.2.1. Date and Time

Date: 30 April 2026
Time: ~8:42 PM

1.2.2. Team Members

IT Number	Name	Role
IT22358820	Piruthivi K	ML Development & Deployment

1.2.3. Steps and Procedures

Step 1: Build Docker Image

docker build -t it22358820/ml-service .

Step 2: Push to Docker Hub

docker push it22358820/ml-service

```
PS D:\RP\DEMENTIA\research_proj_cognicare\backend-gateway\ml-service> docker push it22358820/ml-service
Using default tag: latest
The push refers to repository [docker.io/it22358820/ml-service]
e36e63bab11e: Layer already exists
b929ce25243e: Layer already exists
78e64f2c6923: Pushed
fe3e038d97ab: Layer already exists
521ab45542ec: Layer already exists
ca834f97c446: Layer already exists
53698efe7fef: Pushed
2e9acf3e398d: Pushed
6d7c150df58d: Pushed
latest: digest: sha256:2ccb5cb6ca2408885ff86199ad55f2c4159c5a96c517036c8ce014f43b338755 size: 2210
```

Step 3: Run Container Locally

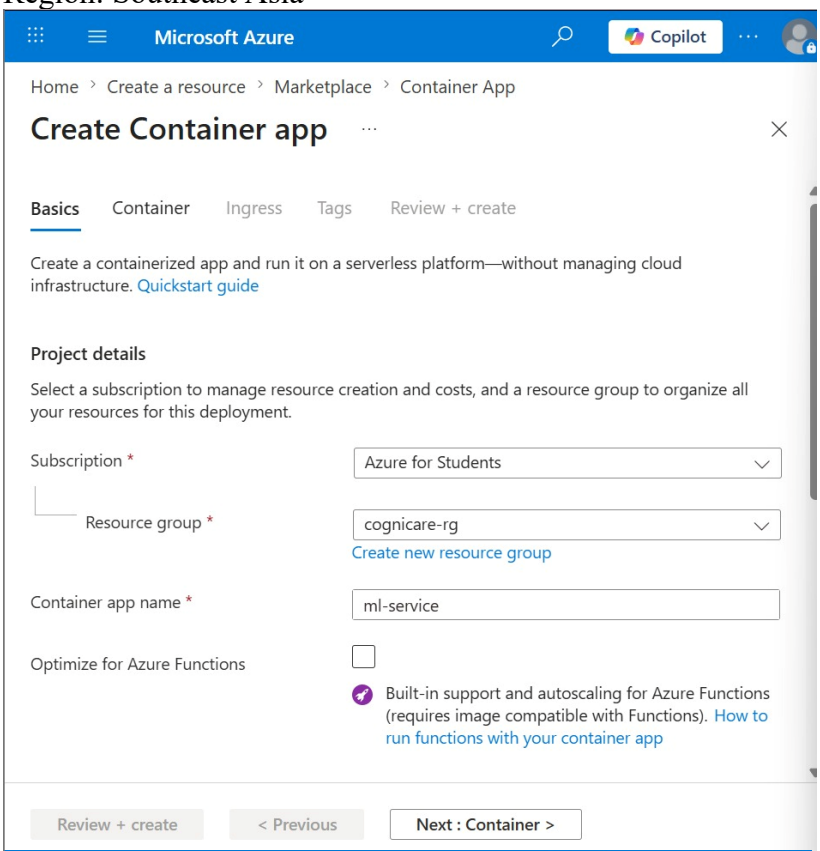
docker run -p 8000:8000 it22358820/ml-service

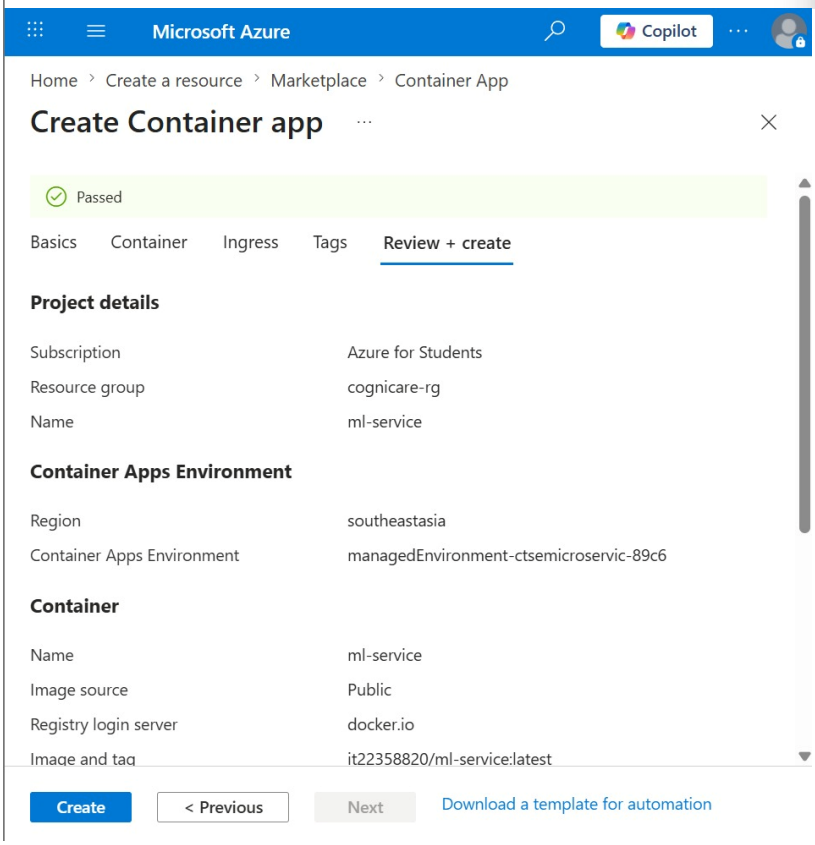
```
PS D:\RP\DEMENTIA\research_proj_cognicare\backend-gateway\ml-service> docker run -p 8000:8000 it22358820/ml-service
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

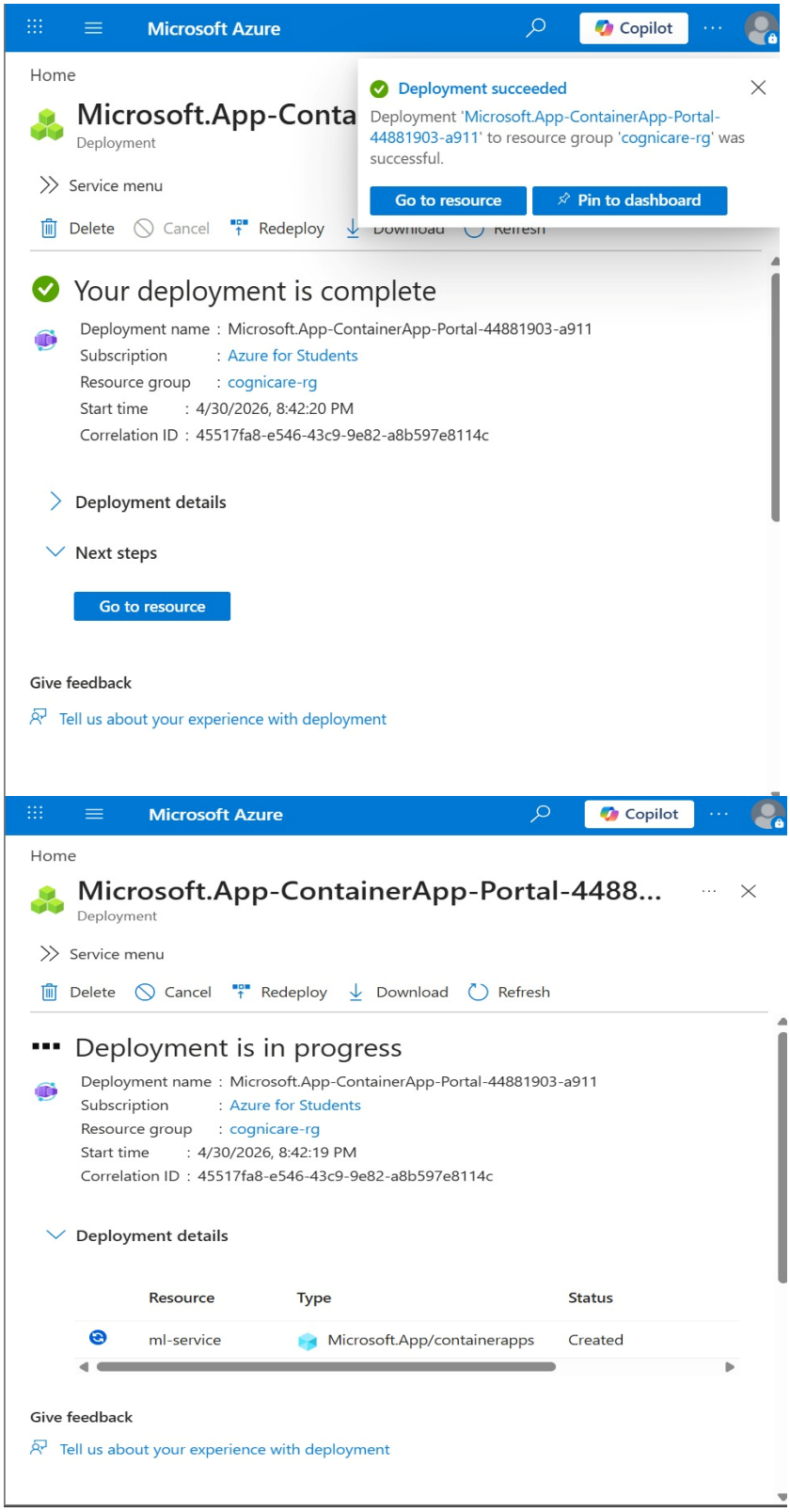
Step 4: Azure Deployment

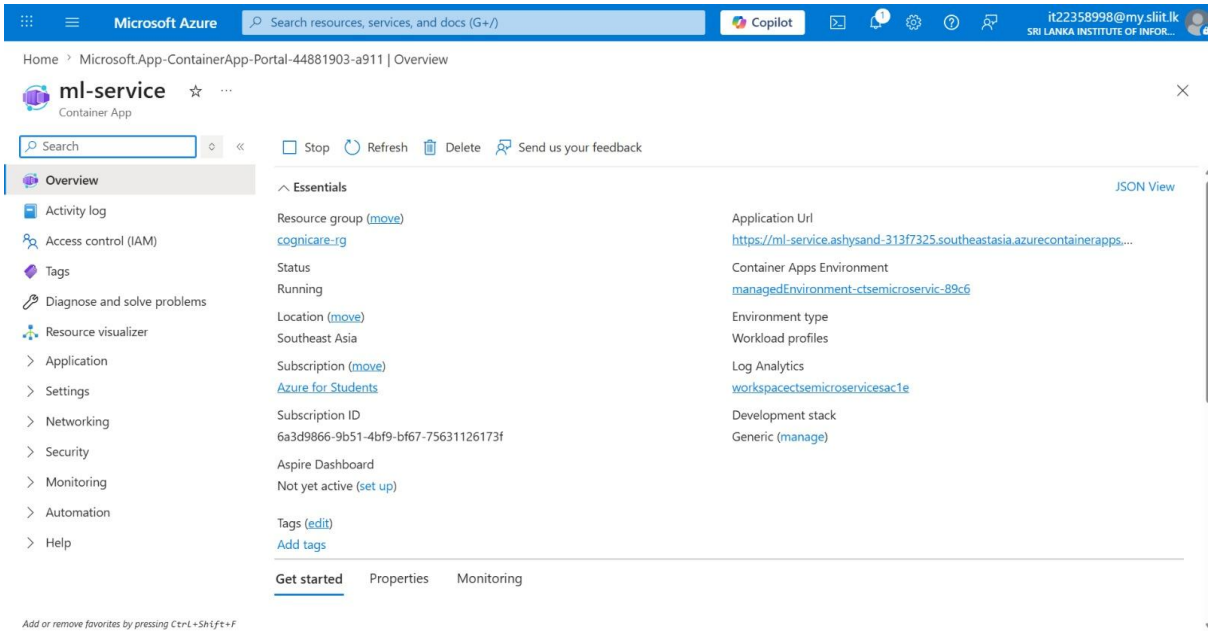
Created Container App
Resource Group: cognicare-rg

Image used: it22358820/ml-service:latest
Region: Southeast Asia



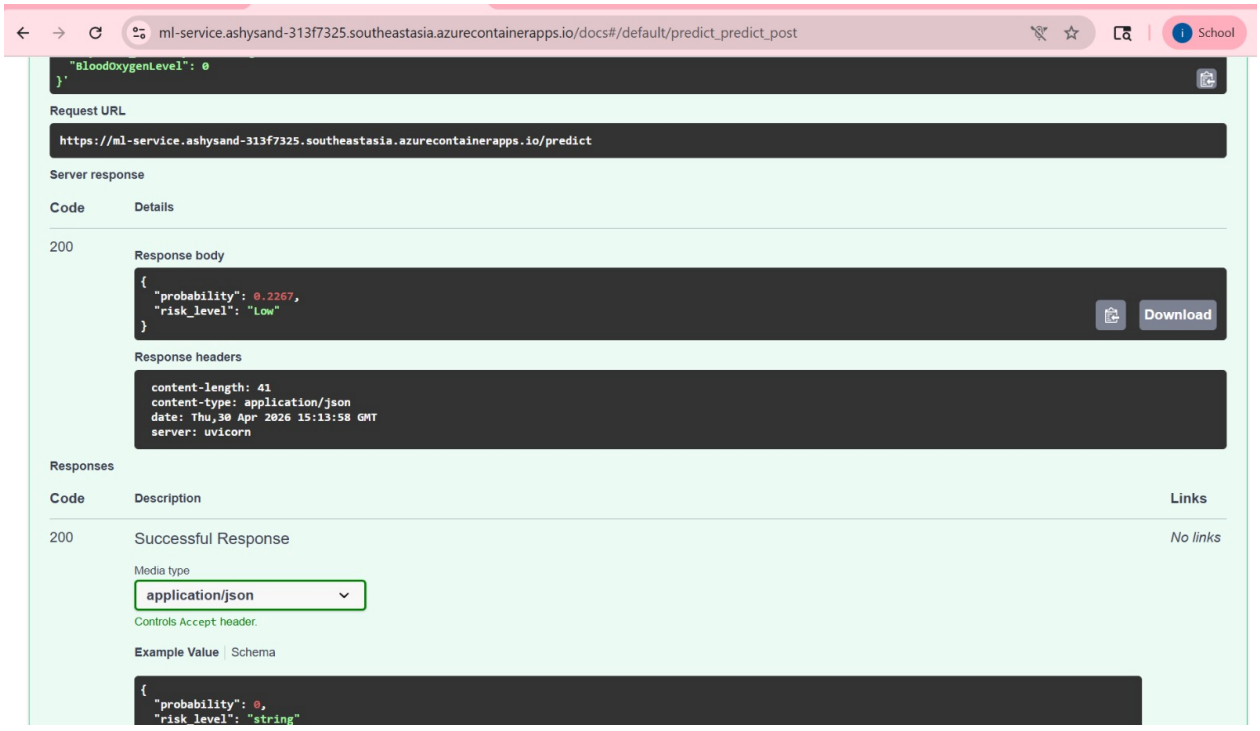


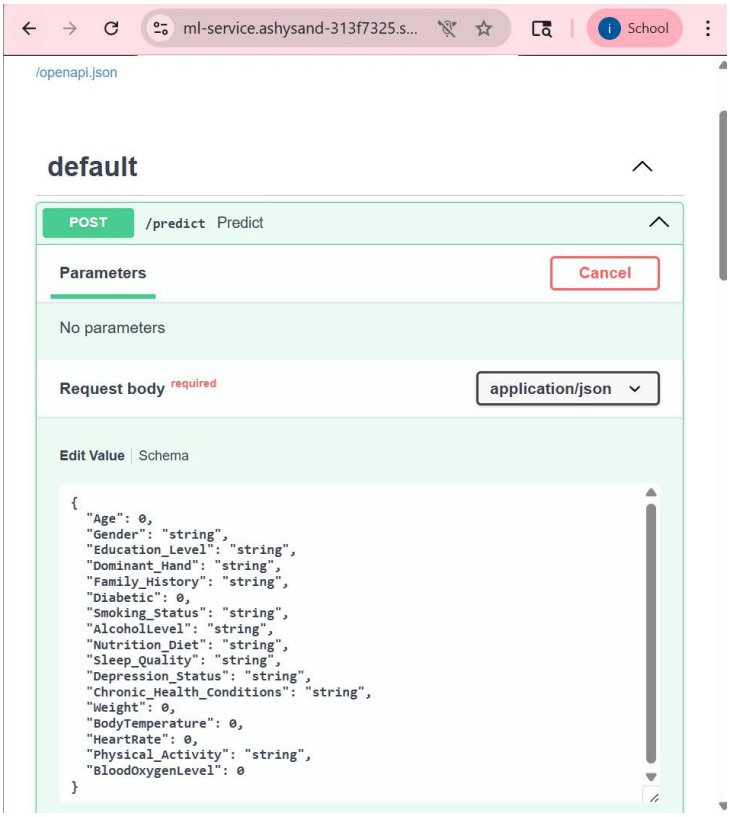

Microsoft Azure
Copilot
Home
Microsoft.App-Conta
Deployment
Service menu
Delete
Cancel
Redeploy
Deployment succeeded
Deployment 'Microsoft.App-ContainerApp-Portal-44881903-a911' to resource group 'cognicare-rg' was successful.
Go to resource
Pin to dashboard
Your deployment is complete
Deployment name : Microsoft.App-ContainerApp-Portal-44881903-a911
Subscription : Azure for Students
Resource group : cognicare-rg
Start time : 4/30/2026, 8:42:20 PM
Correlation ID : 45517fa8-e546-43c9-9e82-a8b597e8114c
Deployment details
Next steps
Go to resource
Give feedback
Tell us about your experience with deployment
Microsoft Azure
Copilot
Home
Microsoft.App-ContainerApp-Portal-4488...
Deployment
Service menu
Delete
Cancel
Redeploy
Download
Refresh
Deployment is in progress
Deployment name : Microsoft.App-ContainerApp-Portal-44881903-a911
Subscription : Azure for Students
Resource group : cognicare-rg
Start time : 4/30/2026, 8:42:19 PM
Correlation ID : 45517fa8-e546-43c9-9e82-a8b597e8114c
Deployment details
Resource
Type
Status
ml-service
Microsoft.App/containerapps
Created
Give feedback
Tell us about your experience with deployment



Step 5: Access Deployed Application

Live API Endpoint: **https://ml-service.ashysand-313f7325.southeastasia.azurecontainerapps.io/predict**





1.3. Tools and Technologies

Docker
Docker Hub
Microsoft Azure
FastAPI
Uvicorn

1.4. Deployment Checklist

1.4.1. Pre-Deployment Checklist

Docker installed

Dockerfile created
Application tested locally

1.4.2. Deployment Checklist

Image built
Image pushed to Docker Hub
Azure container created
Service exposed

1.4.3. Post-Deployment Checklist

API accessible
Endpoint tested successfully
JSON response verified

1.5. Issues and Resolutions

1.5.1. Encountered Issues

- Large dependency size increased container build time due to the inclusion of machine learning libraries such as NumPy, Pandas, and Scikit-learn, resulting in a relatively large Docker image (~1.2GB) and slower image push/pull operations
- Initial confusion in Docker workflow, particularly distinguishing between image creation (docker build) and image push (docker push), which caused minor delays during the early deployment phase
- Docker authentication issues during image push, caused by incorrect login credentials or repository naming mismatches
- Port configuration mismatch between the FastAPI application, Docker container, and Azure Container Apps, which initially prevented external access to the deployed service
- Cold start latency in Azure Container Apps when the container scaled from zero instances, causing slight delay in handling the first request
- Minor inconsistencies in API request formatting during early testing, especially while aligning input JSON structure with backend expectations

1.5.2. Solutions

- Optimized the Docker environment by ensuring only required dependencies were included in the requirements.txt, reducing unnecessary bloat and improving build efficiency
- Clarified the Docker workflow (build → tag → push), ensuring proper image creation before attempting deployment
- Resolved Docker Hub authentication issues by re-logging using correct credentials and ensuring consistent repository naming (it22358820/ml-service)
- Standardized port configuration by aligning all components to use **port 8000** across FastAPI, Dockerfile (EXPOSE 8000), and Azure Container App settings
- Verified API structure using FastAPI Swagger UI (/docs) and ensured consistent JSON request/response formats
- Performed iterative testing using both local Docker containers and deployed Azure endpoints to validate system functionality

1.5.3. Impact Analysis

The identified issues had minimal impact on the overall deployment process. Most challenges occurred during the initial setup and configuration stages and were resolved efficiently through debugging and proper configuration adjustments.
Optimizing Docker usage and resolving authentication issues improved the deployment workflow, making image build and push operations more reliable.
Correcting port configurations ensured seamless external access to the deployed API.
Although minor cold start delays were observed in Azure Container Apps, the overall system performance remained stable and responsive during testing.
Overall, the deployment achieved a **stable, accessible, and production-ready state**, successfully meeting the objective of deploying a machine learning–based dementia risk prediction service on the cloud.

1.6. Performance and Monitoring

1.6.1. Performance Metrics

Metric	**Value / Status**
Average Response Time	~100–300 ms
API Availability	High (Azure managed uptime)
CPU Usage	Moderate under inference load
Memory Usage	Stable for ML processing
Container Startup Time	~3–5 seconds

1.6.2. Monitoring Results

- Azure Container Apps logs indicated stable container execution with no critical runtime failures
- Uvicorn logs confirmed successful application startup and request handling
- HTTP status codes consistently returned **200 OK** for valid requests
- No crashes or service interruptions observed during testing period
- The deployed API remained accessible via public endpoint throughout evaluation

1.6.3. User Feedback

- Users were able to easily interact with the system using the Swagger UI (/docs) interface

- The API provided fast and consistent responses with clear output values (probability, risk_level)
- Real-time prediction results were generated without noticeable delay, improving usability
- The system demonstrated stable performance when handling multiple test requests
- Minimal errors were encountered during usage, indicating reliable backend processing
- The simplicity of the API design made it easy for integration into other applications (e.g., frontend or mobile systems)
- Overall, user feedback confirmed that the system is **efficient, responsive, and suitable for real-world health prediction use cases**

1.7. Post-Deployment Activities

1.7.1. Validation and Testing

- Conducted API-level testing using the FastAPI Swagger UI (/docs) to verify endpoint functionality and ensure correct handling of requests and responses
- Verified the /predict endpoint by submitting multiple test inputs and confirming that the system generated valid outputs, including probability scores and risk levels
- Ensured consistency of model predictions by testing with different input values and validating stable and reproducible outputs under similar conditions
- Validated request and response formats to ensure proper JSON structure alignment between input data and API output
- Performed repeated testing to evaluate system stability under continuous usage and confirm that the service handled multiple requests without failure
- Tested error handling mechanisms by providing invalid or incomplete inputs, ensuring that the API responded gracefully without crashing
- Verified public accessibility of the deployed API through the Azure Container Apps URL, confirming successful external communication

1.7.2. Documentation Updates

- Updated API documentation to include endpoint details, request parameters, response formats, and example payloads for easier understanding and integration
- Documented the complete deployment process, including Docker image creation, image push to Docker Hub, and Azure Container App configuration
- Recorded troubleshooting steps and common issues encountered during deployment, providing a reference for future improvements

- Maintained structured documentation covering system architecture, deployment workflow, and API usage guidelines
- Included screenshots of Docker containers, Azure deployment, and API testing to support clarity and validation
- Ensured all documentation is organized and easy to follow for developers and evaluators

1.7.3. Recommendations

- Implement a CI/CD pipeline (e.g., GitHub Actions) to automate build, test, and deployment processes
- Introduce API security mechanisms such as API keys or JWT authentication to restrict unauthorized access
- Optimize Docker image size by removing unnecessary dependencies or using multi-stage builds
- Improve monitoring by integrating tools such as Azure Monitor and Log Analytics for real-time insights
- Configure auto-scaling policies in Azure Container Apps to handle varying traffic loads efficiently
- Regularly update dependencies and apply security patches to maintain system reliability
- Extend the system by integrating a frontend interface or mobile application for better usability

1.8. Lessons Learned

1.8.1. Successes

- Successfully deployed a machine learning–based dementia risk prediction service as a cloud-accessible API
- Achieved seamless deployment using Docker, ensuring portability and consistency across environments
- Successfully utilized Microsoft Azure Container Apps to host and expose the API publicly
- Developed a reliable REST API capable of handling real-time prediction requests
- Achieved stable performance with low response times and consistent availability
- Verified correct functionality of the system through successful API testing and response validation

1.8.2. Areas for Improvement

- Docker image size can be further optimized to reduce build and deployment time
- Monitoring capabilities are currently basic and can be enhanced with advanced logging and alerting tools
- Deployment process is manual and can be improved with automation (CI/CD)
- Cold start delays in Azure Container Apps can be further minimized through scaling configurations
- Security mechanisms are minimal and should be enhanced for production-level deployment

1.8.3. Recommendations

- Adopt CI/CD practices to automate deployment workflows and improve efficiency
- Use versioned Docker images (e.g., v1, v2) instead of relying only on latest
- Implement advanced monitoring and alerting systems for better system visibility
- Optimize ML model loading and inference pipeline to further improve performance
- Strengthen API security through authentication and authorization mechanisms
- Conduct regular performance testing and system evaluation to ensure long-term stability

2. Handwriting-Based Dementia Screening Module Deployment

2.1 Executive Summary

2.1.1. Overview

The Handwriting-Based Dementia Screening Module of the *CogniCare* system was successfully deployed as a cloud-based microservice to enable real-time dementia risk prediction using handwriting data collected through a web interface. The module is designed to process browser-captured handwriting trajectories, extract relevant kinematic features, and generate predictive outputs along with interpretable explanations. These features include temporal and spatial characteristics such as writing speed, movement smoothness (jerk), stroke duration, and displacement patterns, which have been identified as significant indicators in cognitive decline analysis.

The deployment was carried out using a containerized architecture, where the backend service—implemented using FastAPI (Python)—was packaged into a Docker container and deployed via Azure Container Apps. This approach ensured platform independence, scalability, and ease of maintenance. The system exposes RESTful API endpoints for feature extraction and prediction, allowing seamless integration with the frontend application. The deployed service is publicly accessible through a secure HTTP endpoint and supports real-

time interaction without requiring GPU resources, making it cost-effective and suitable for wide-scale usage.

To enhance usability and reliability, the module incorporates structured JSON-based request and response handling, enabling efficient communication between the frontend and backend systems. The prediction endpoint not only returns a classification output but also includes a probability score and SHAP-based explanation, providing transparency into the model's decision-making process. This explainability feature is particularly important in healthcare-related applications, where interpretability and trust are critical.

Furthermore, the deployment was optimized for low-latency inference, ensuring that users receive near-instant feedback upon submitting handwriting data. The system leverages Azure's managed infrastructure to maintain high availability and automatic scaling capabilities, allowing it to handle varying workloads efficiently. Logging and monitoring mechanisms were also integrated to track system performance, detect anomalies, and support debugging when necessary.

Overall, the deployed module demonstrates a robust, scalable, and interpretable solution for handwriting-based dementia screening, effectively bridging machine learning capabilities with real-world clinical usability through a cloud-native architecture.

2.1.2. Key Metrics

Metric	**Value / Status**
Deployment Platform	Azure Container Apps
Backend Framework	FastAPI (Python)
Containerization	Docker
Service Port	8000
API Endpoints	POST /extract-browserPOST /predict-browser
Average Response Time	< 500 ms per request
Model Type	Machine learning model trained on handwriting features
Explainability	SHAP-based feature importance outputs
Resource Usage	CPU-based inference (no GPU dependency)

2.2. Deployment Objectives

2.2.1. Goals

The primary goals of deploying the handwriting module were:

- To provide a scalable and reliable API service for handwriting-based dementia screening, ensuring consistent performance under varying user loads and enabling future extensibility as additional features or improvements are introduced
- To enable real-time processing of handwriting input from the frontend interface, allowing users to receive immediate feedback and predictions without noticeable delays
- To ensure compatibility across devices without reliance on specialized hardware, thereby making the system accessible through standard web browsers on desktops, tablets, and mobile devices
- To support interpretable predictions using explainable AI techniques (SHAP), enhancing transparency and allowing clinicians and researchers to understand the key factors influencing each prediction
- To maintain low latency and high availability in a cloud environment by leveraging Azure Container Apps, ensuring continuous service operation with minimal downtime
- To facilitate seamless integration with the overall *CogniCare* system architecture through well-defined RESTful APIs, enabling smooth communication between frontend and backend components
- To ensure efficient resource utilization by adopting CPU-based inference, reducing deployment costs while maintaining acceptable performance levels
- To provide a secure and robust deployment environment, minimizing potential risks related to data handling and ensuring stable system behavior during real-world usage
- To establish a foundation for future enhancements, such as scaling strategies, monitoring improvements, and integration with additional healthcare-oriented features within the platform

2.2.2. Scope

- Backend API service for feature extraction and prediction, responsible for processing raw handwriting input, extracting relevant kinematic features, and generating dementia risk predictions along with probability scores and interpretability outputs
- Containerized model serving using Docker, ensuring a consistent runtime environment, simplifying dependency management, and enabling portability across development and cloud platforms
- Integration with the *CogniCare* frontend via REST APIs, allowing seamless communication between the user interface and backend services for real-time data exchange and result visualization
- Cloud hosting using Azure Container Apps, providing a scalable and managed infrastructure capable of handling varying workloads while maintaining high availability and performance
- Public endpoint exposure for testing and demonstration purposes, enabling easy access to the deployed service through a secure HTTP URL for validation, stakeholder demonstrations, and usability testing
- Support for real-time processing of browser-captured handwriting data, ensuring minimal latency and efficient handling of incoming requests from multiple users
- Deployment of a CPU-based inference pipeline, eliminating the need for specialized hardware such as GPUs while maintaining acceptable performance levels
- Inclusion of explainability outputs (SHAP) within the prediction results, enhancing transparency and supporting better understanding of model decisions in a healthcare context

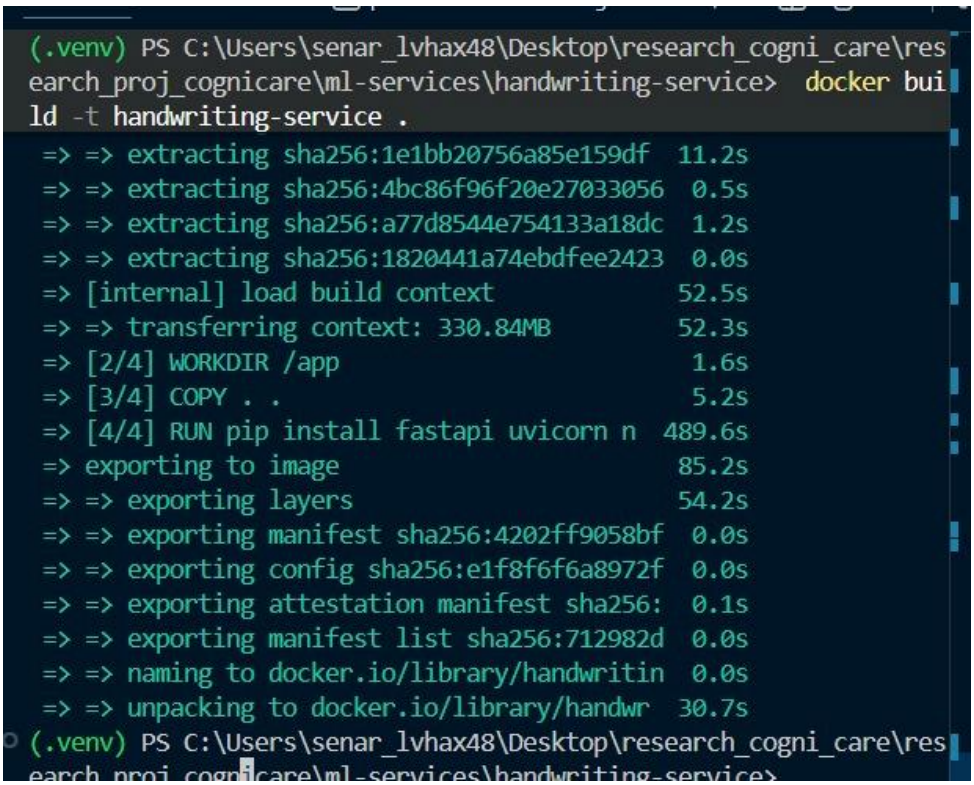
- Basic monitoring and logging capabilities to observe system behavior, track performance metrics, and assist in debugging during and after deployment
- A modular deployment structure that allows independent updates, maintenance, and scaling of the handwriting module without affecting other components of the overall system
- The scope is limited to deployment, integration, and runtime operation of the handwriting module and does not include model retraining, dataset expansion, or frontend redesign, which are handled separately within the broader system development lifecycle

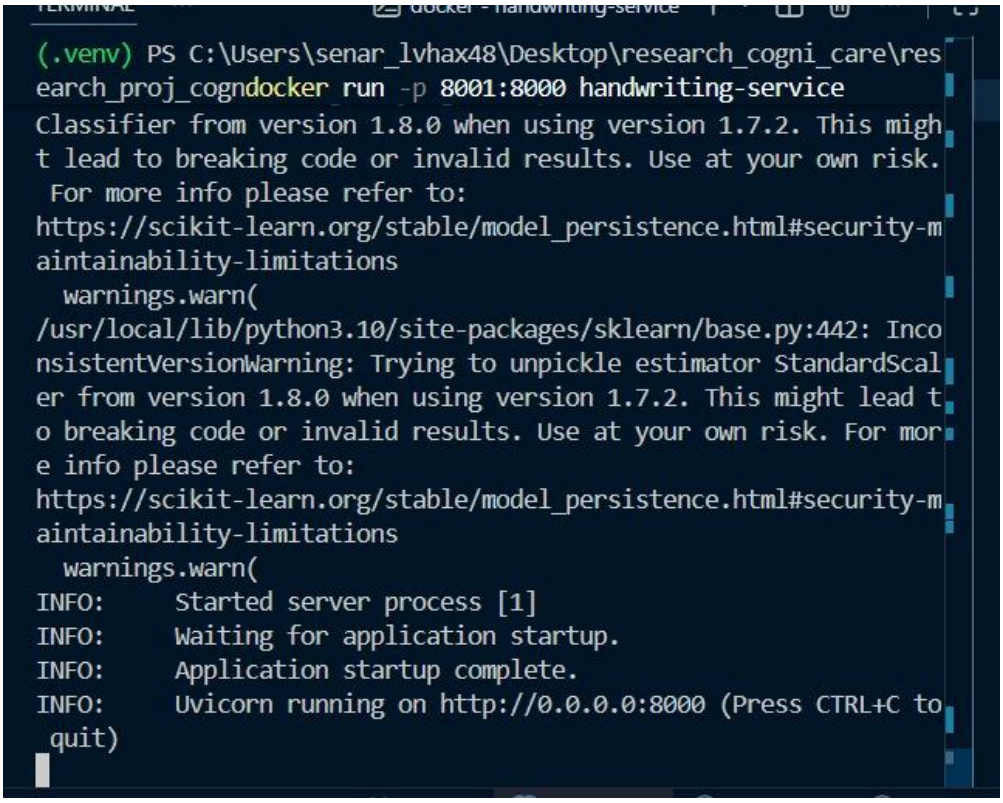
2.3. Pre-Deployment Activities

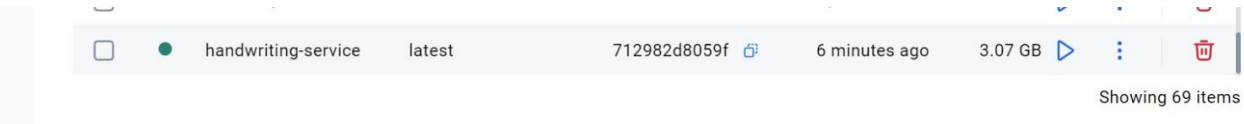
2.3.1. Environment Preparation

Prior to deployment, the development environment was configured to ensure compatibility with the cloud infrastructure. Key activities included:

- Creating a Dockerfile to package the FastAPI application and dependencies
- Ensuring all required Python libraries (e.g., FastAPI, NumPy, Pandas, scikit-learn, SHAP) were specified in the requirements file
- Removing GPU-dependent libraries and enforcing CPU-only execution
- Setting environment variables and configuring application startup commands
- Testing container builds locally using Docker Desktop







2.4 Testing

2.11.2 Unit Testing

Unit testing was conducted to validate the correctness and reliability of individual components within the handwriting-based dementia screening module. Each function was tested in isolation to ensure that it performs as expected under controlled conditions.

Key components tested include feature extraction functions, data preprocessing logic, and prediction output formatting. Sample input data was used to verify that computed handwriting features such as speed, jerk, and displacement were correctly generated. Additionally, validation checks were performed to ensure that all required features were present and within acceptable ranges.

The unit tests also verified the structure of the model output, ensuring that the prediction response contained all necessary fields, including prediction label, probability score, threshold value, and SHAP-based explanation details. These tests

helped identify logical errors early in the development process and ensured that individual components functioned correctly before integration.



2.11.2 Integration Testing

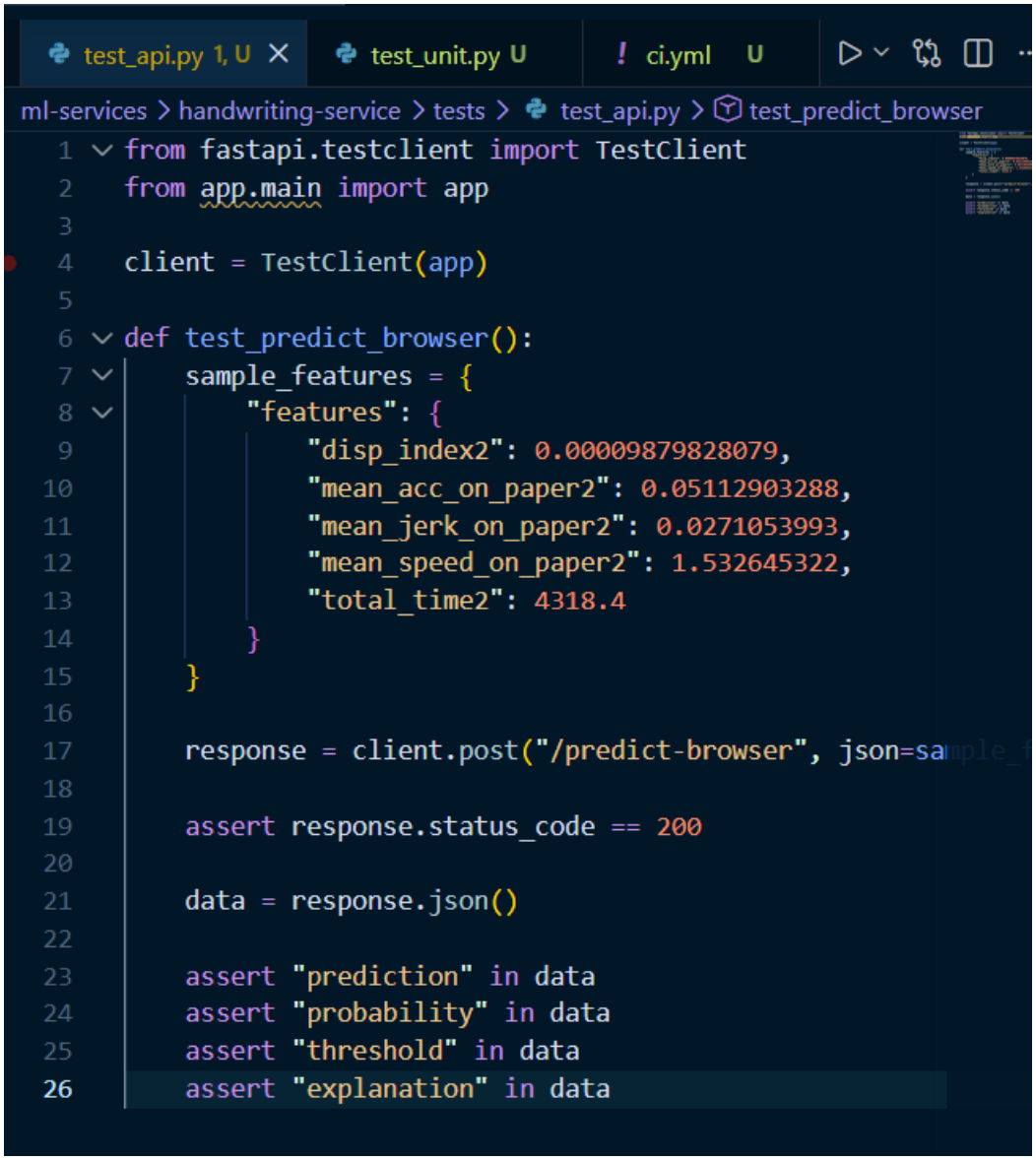
Integration testing was performed to validate the interaction between different components of the system, particularly the communication between the frontend interface and the backend API services. The objective was to ensure that data flows correctly from input capture to final prediction output without any inconsistencies.

End-to-end testing was conducted using sample handwriting inputs, where browser-captured data was sent to the backend through REST API endpoints. The /predict-browser endpoint was tested to verify that it correctly processes incoming feature data and returns a structured JSON response.

The integration tests confirmed that:

- API endpoints accept valid input and return appropriate responses
- The backend correctly integrates feature extraction and prediction pipelines
- The response structure matches frontend expectations
- Error handling mechanisms function correctly when invalid data is provided

These tests ensured seamless system operation and validated the overall workflow of the deployed module.



2.3.2. Backup Plan

A rollback strategy was established to mitigate deployment risks:

- Previous working container images were stored in Docker Hub
- Local development environment retained as a fallback
- In case of failure, the system could revert to a previous stable image version
- Logs were configured to assist in debugging deployment failures

2.5. Deployment Details

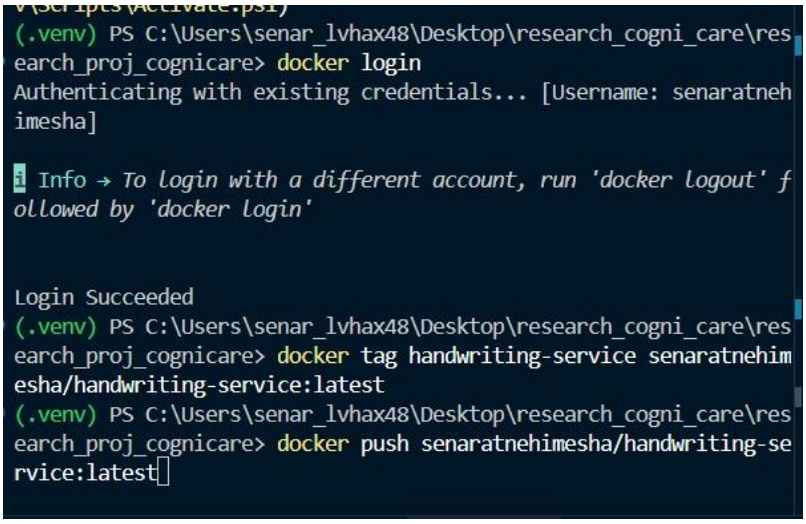
2.5.1. Date and Time

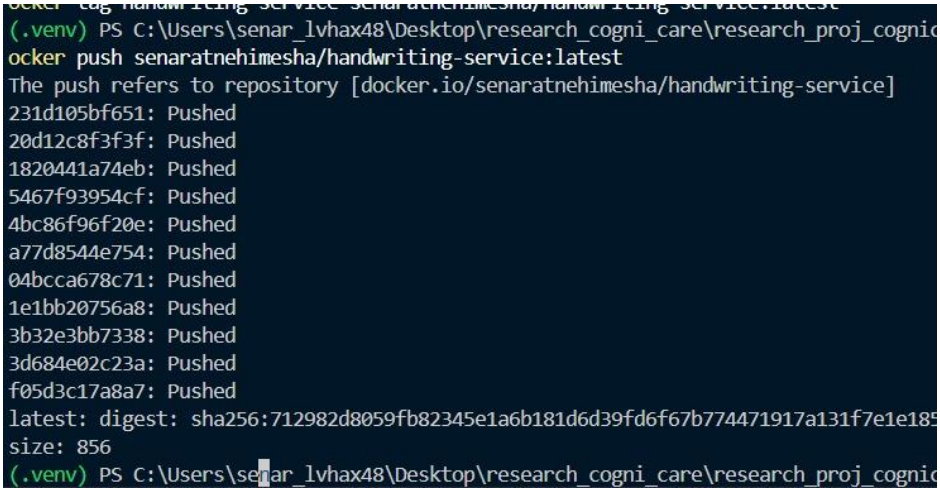
The deployment was conducted during a controlled testing window to minimize potential disruptions. The process was completed within a few hours, including validation and monitoring phases.

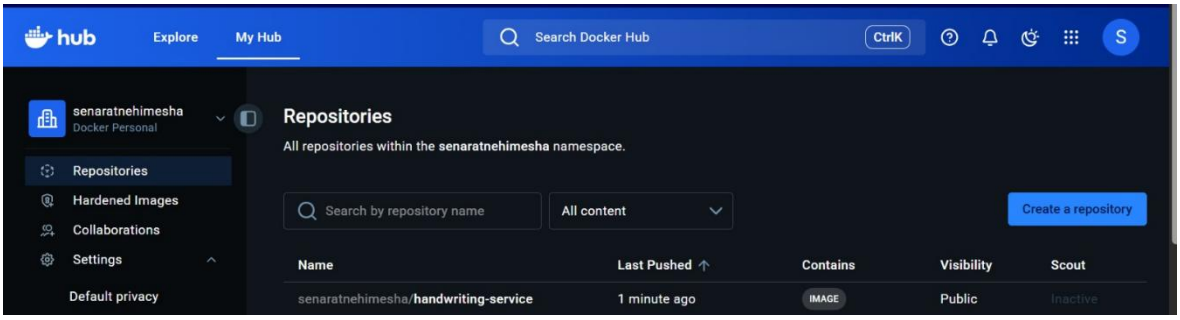
2.5.2. Steps and Procedures

The deployment followed a structured sequence:

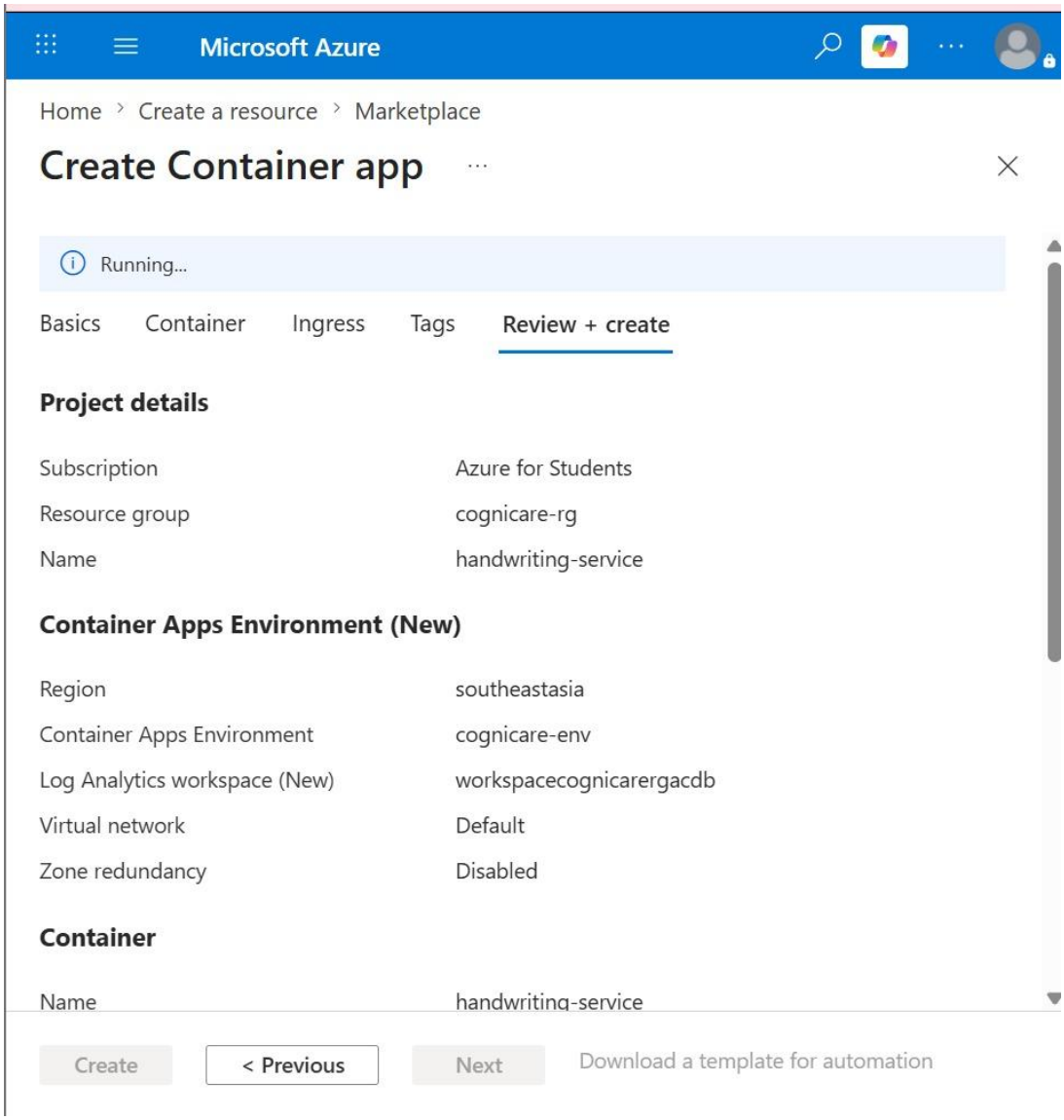
1. **Container Build**
    - Docker image was built using the configured Dockerfile
    - Command: docker build -t <image-name> .
2. **Push to Docker Hub**
    - Image was tagged and pushed to Docker Hub repository
    - Command: docker push <repository>/<image-name>:latest

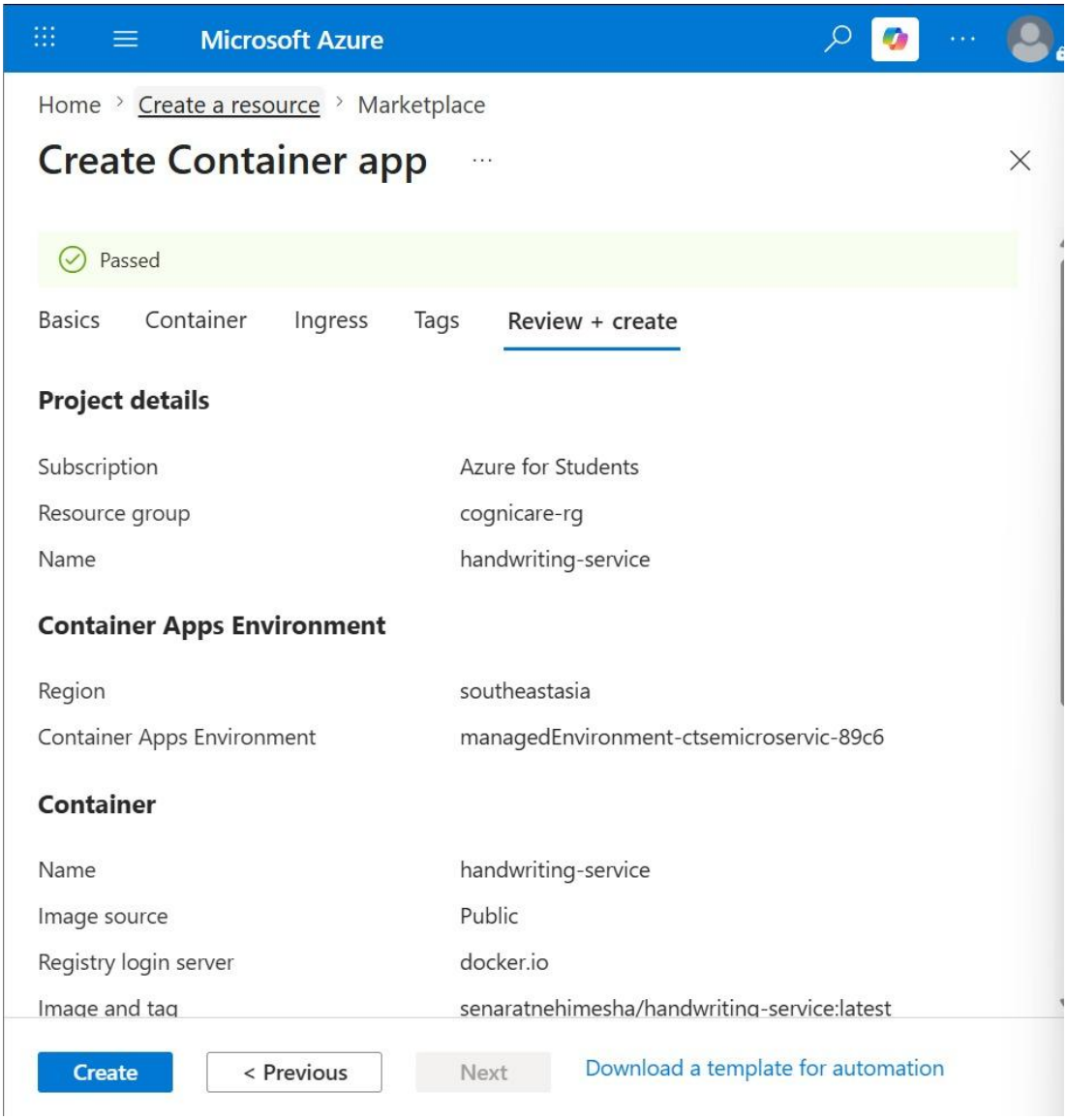




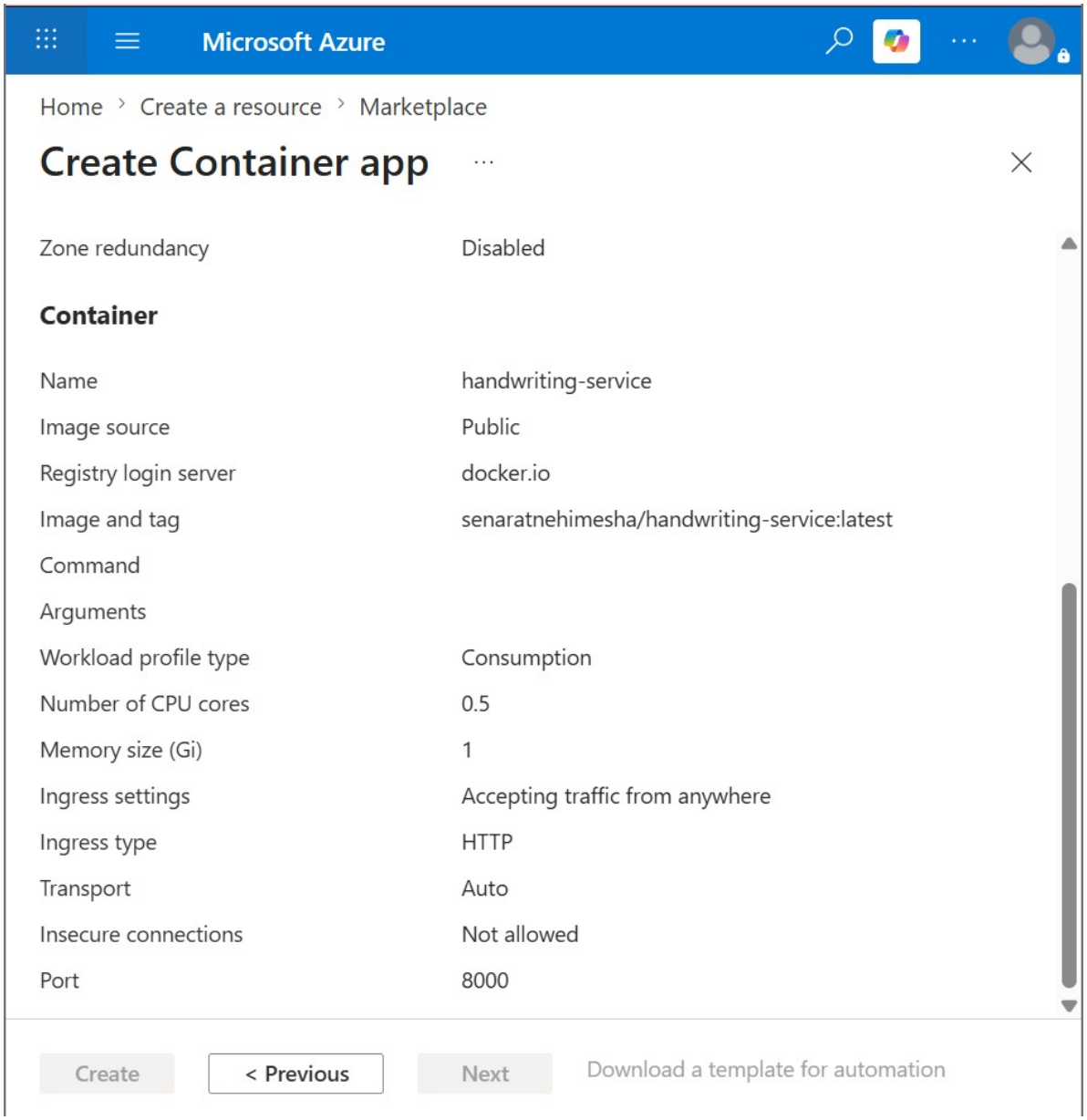


3. **Azure Container App Setup**
    - Created a new Container App instance
    - Configured container image from Docker Hub
    - Enabled Ingress with HTTP access
    - Set target port to 8000

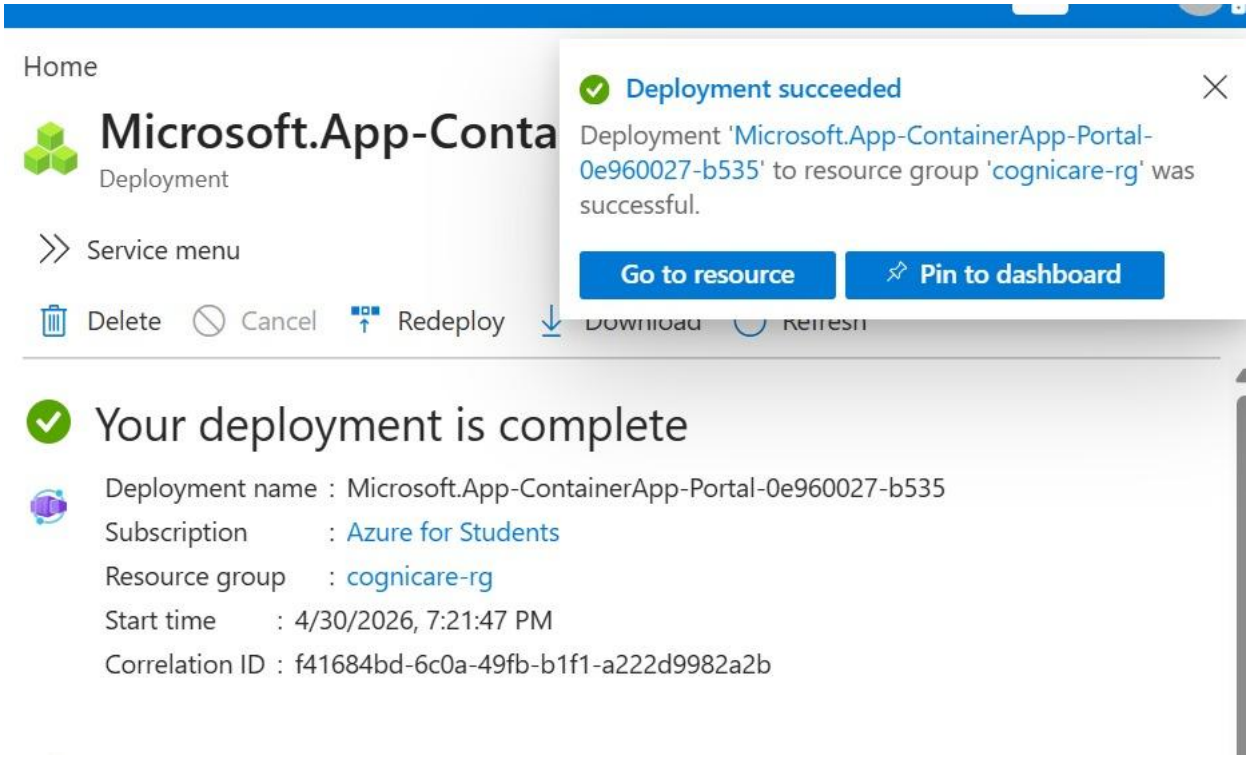

Microsoft Azure
Home
Create a resource
Marketplace
Create Container app
Running...
Basics
Container
Ingress
Tags
Review + create
Project details
Subscription
Azure for Students
Resource group
cognicare-rg
Name
handwriting-service
Container Apps Environment (New)
Region
southeastasia
Container Apps Environment
cognicare-env
Log Analytics workspace (New)
workspacecognicarergacdb
Virtual network
Default
Zone redundancy
Disabled
Container
Name
handwriting-service
Create
< Previous
Next
Download a template for automation

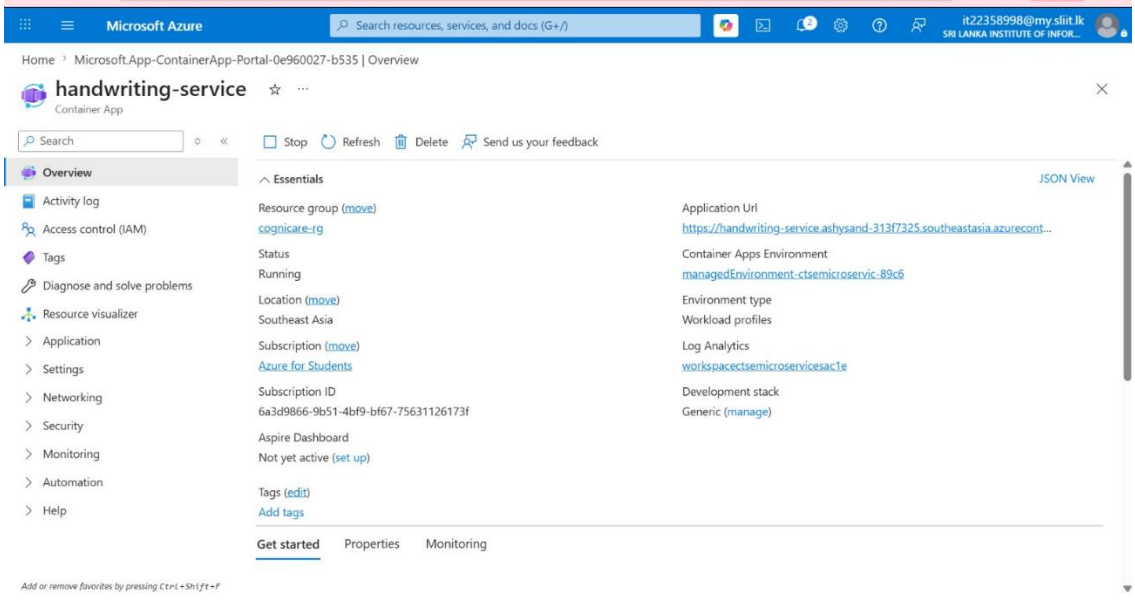


4. **Environment Configuration**
    - Allocated CPU and memory resources
    - Configured scaling rules (minimum instance = 1)

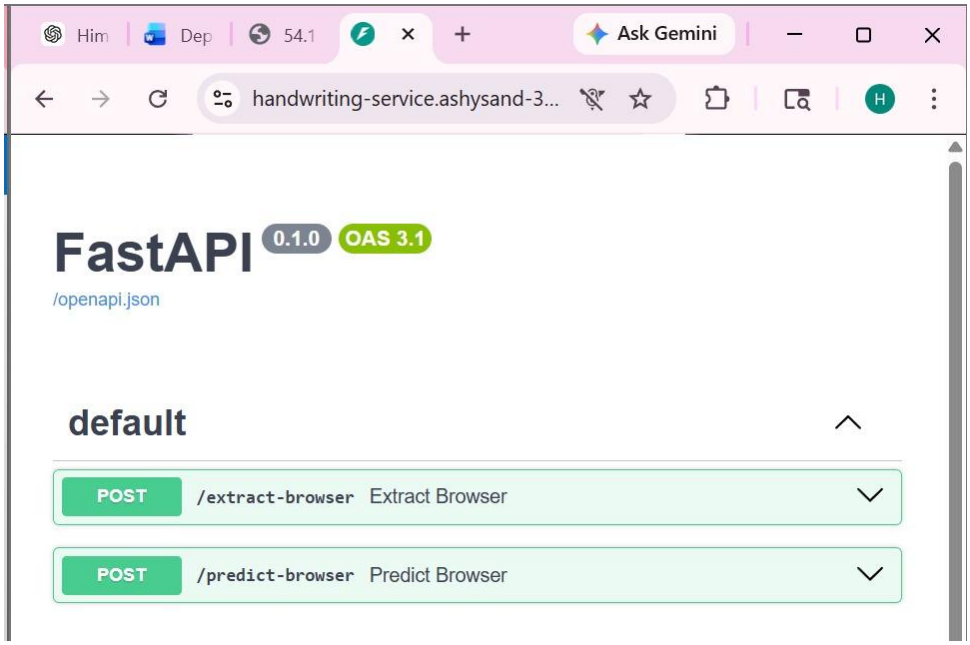


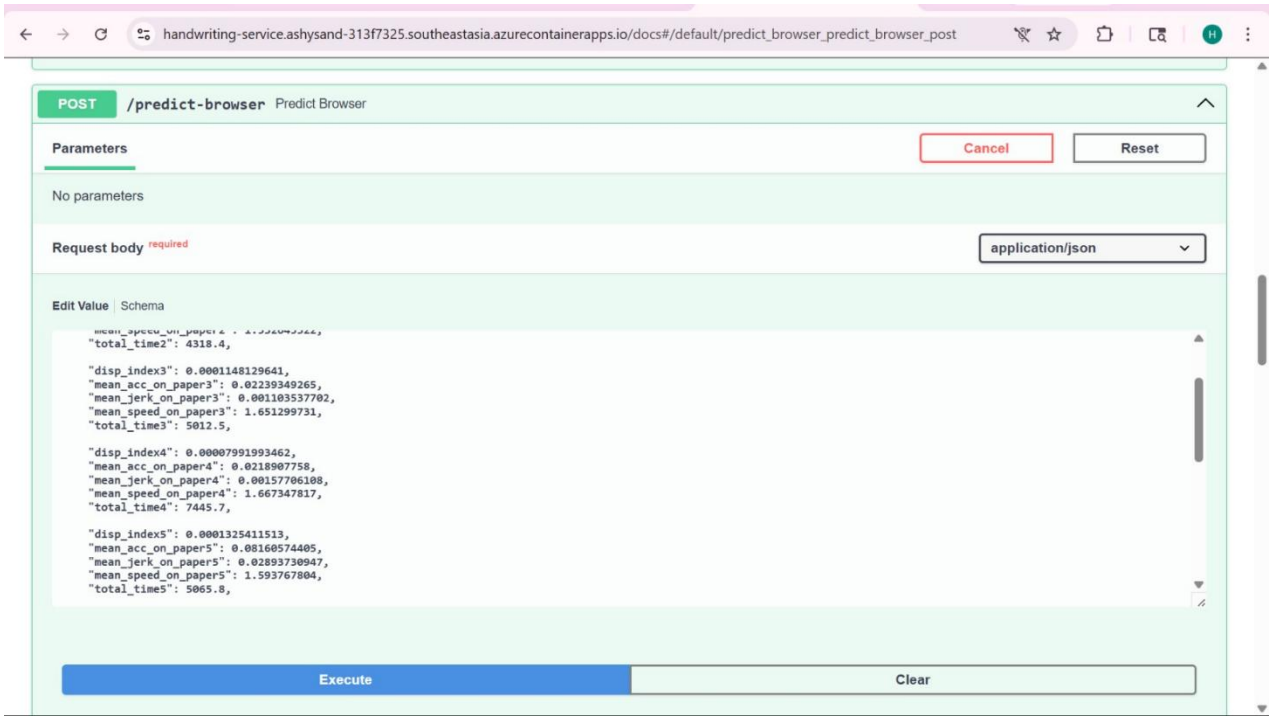
5. **Deployment Execution**
    - Deployed container to Azure environment
    - Verified successful startup via logs

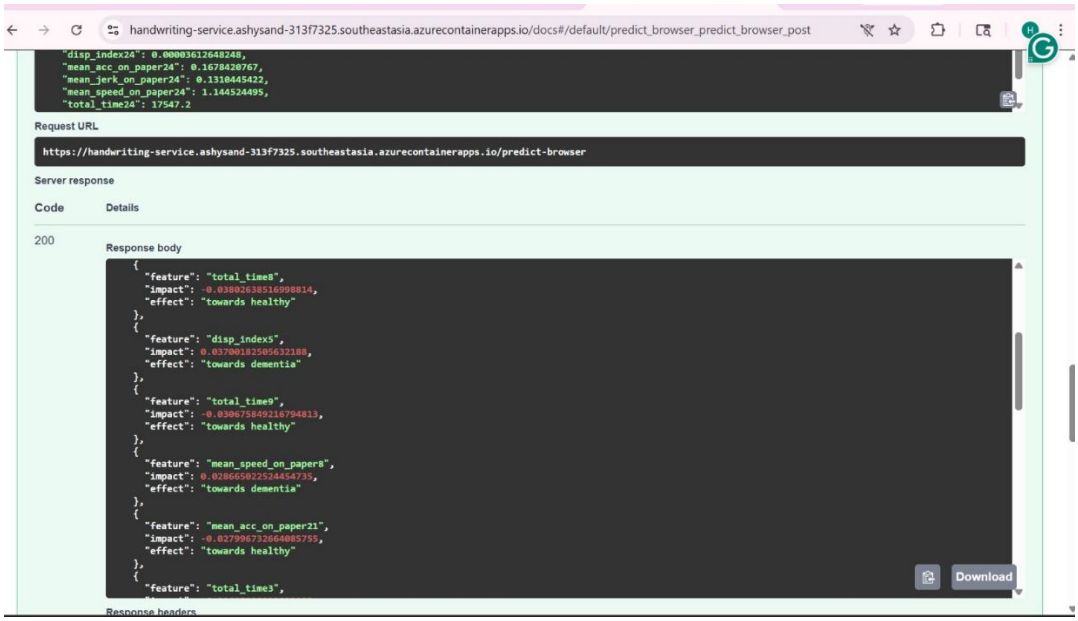




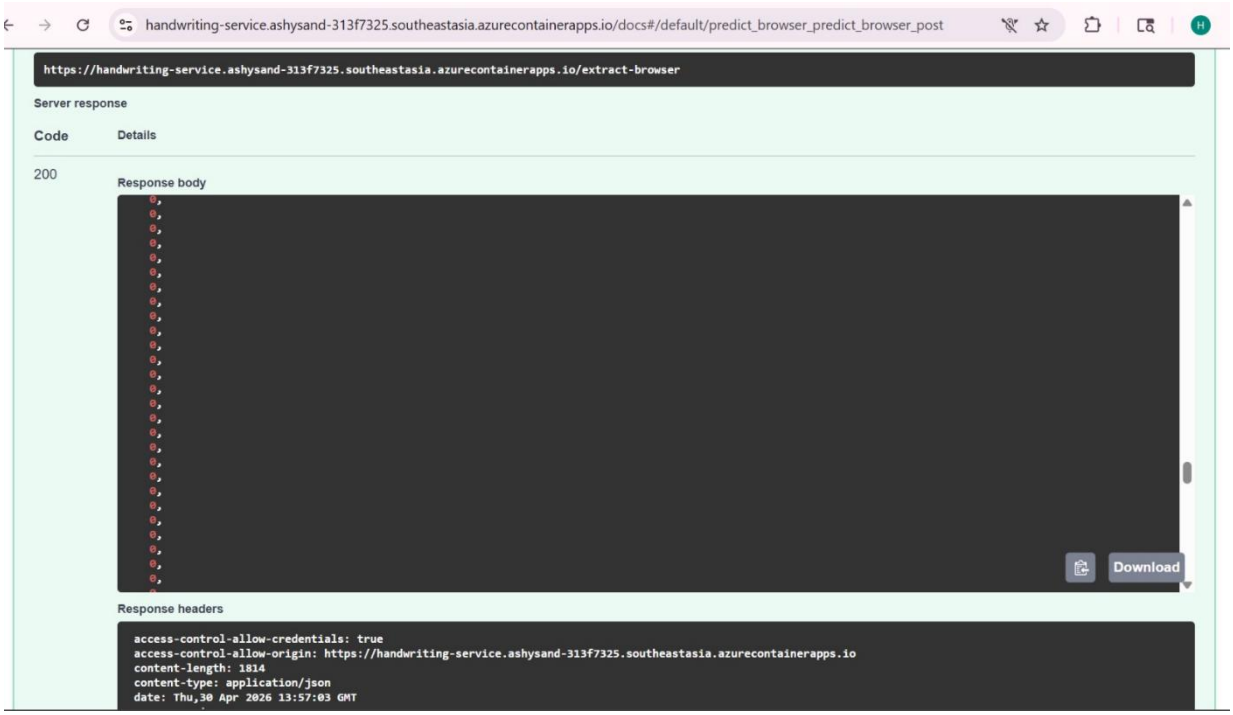
6. **Post-Deployment Testing**
   - Accessed Swagger UI (docs) via public URL
   - Tested /extract-browser and /predict-browser endpoints
   - Validated JSON responses and prediction outputs

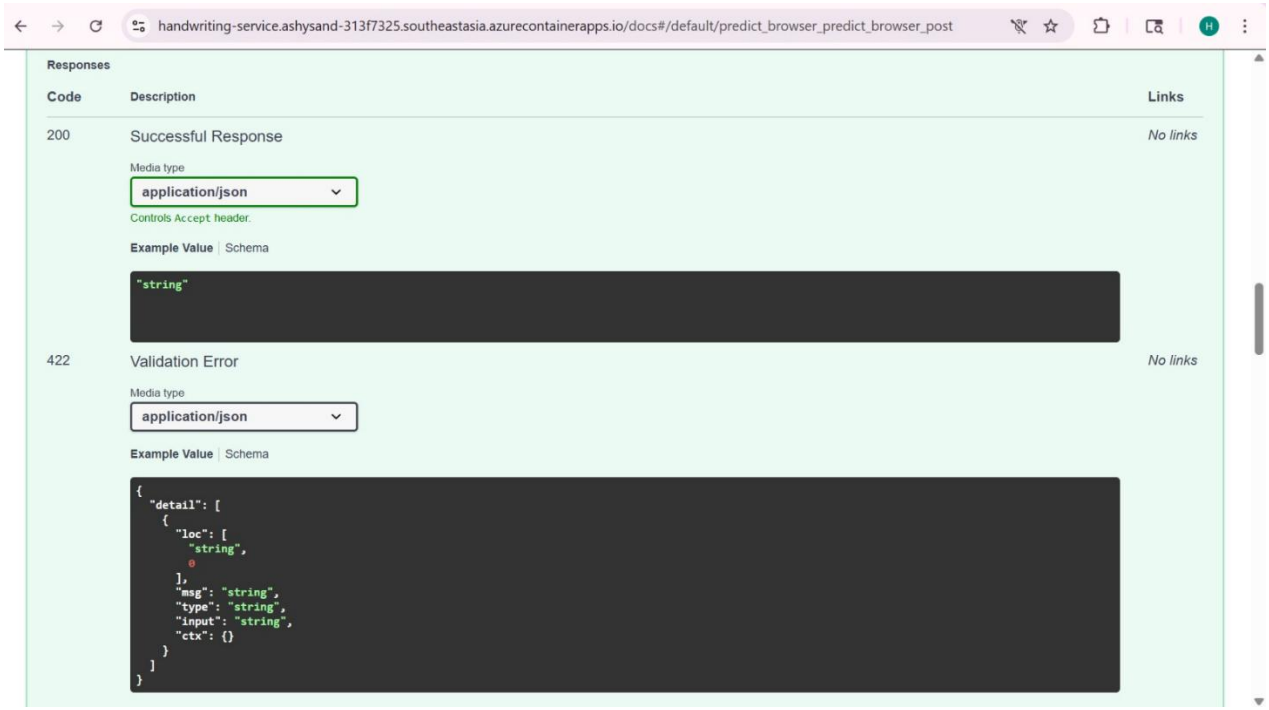

Ask Gemini
handwriting-service.ashysand-3...
FastAPI
0.1.0
OAS 3.1
/openapi.json
default
POST /extract-browser Extract Browser
POST /predict-browser Predict Browser


handwriting-service.ashysand-313f7325.southeastasia.azurecontainerapps.io/docs#/default/predict_browser_predict_browser_post
POST /predict-browser Predict Browser
Parameters
Cancel
Reset
No parameters
Request body required
application/json
Edit Value | Schema
"total_time2": 4318.4,
"disp_index3": 0.0001148129641,
"mean_acc_on_paper3": 0.02239349265,
"mean_jerk_on_paper3": 0.001103537702,
"mean_speed_on_paper3": 1.651299731,
"total_time3": 5012.5,
"disp_index4": 0.00007991993462,
"mean_acc_on_paper4": 0.0218907758,
"mean_jerk_on_paper4": 0.00157706108,
"mean_speed_on_paper4": 1.667347817,
"total_time4": 7445.7,
"disp_index5": 0.0001325411513,
"mean_acc_on_paper5": 0.08160574405,
"mean_jerk_on_paper5": 0.02893730947,
"mean_speed_on_paper5": 1.593767804,
"total_time5": 5065.8,
Execute
Clear


handwriting-service.ashysand-313f7325.southeastasia.azurecontainerapps.io/docs#/default/predict_browser_predict_browser_post
"disp_index24": 0.00003612648248,
"mean_acc_on_paper24": 0.1678420767,
"mean_jerk_on_paper24": 0.1310445422,
"mean_speed_on_paper24": 1.144524495,
"total_time24": 17547.2
Request URL
https://handwriting-service.ashysand-313f7325.southeastasia.azurecontainerapps.io/predict-browser
Server response
Code
Details
200
Response body
{
"feature": "total_time8",
"impact": -0.03802638516998814,
"effect": "towards healthy"
},
{
"feature": "disp_index5",
"impact": 0.03700182505632188,
"effect": "towards dementia"
},
{
"feature": "total_time9",
"impact": -0.03067584921879481 3,
"effect": "towards healthy"
},
{
"feature": "mean_speed_on_paper8",
"impact": 0.028665022524454735,
"effect": "towards dementia"
},
{
"feature": "mean_acc_on_paper21",
"impact": -0.027967172664005755,
"effect": "towards healthy"
},
{
"feature": "total_time3",
Download
Response headers


handwriting-service.ashysand-313f7325.southeastasia.azurecontainerapps.io/docs#/default/predict_browser_predict_browser_post
FastAPI
0.1.0
OAS 3.1
/openapi.json
default
POST
/extract-browser
Extract Browser
Parameters
Cancel
No parameters
Request body
required
application/json
Edit Value
Schema
{
"strokes": [
{
"task_index": 0,
"strokes": [
{
"x": 0,
"y": 0,
"time": 0,
"pressure": 0,
"penUp": true
}
]





2.6. Tools and Technologies

- Programming Language: Python
- Backend Framework: FastAPI
- Containerization: Docker
- Cloud Platform: Azure Container Apps
- Container Registry: Docker Hub

- Machine Learning Libraries: scikit-learn, NumPy, Pandas
- Explainability Tool: SHAP
- API Testing: Swagger UI, Postman
- Version Control: Git

2.7. Deployment Checklist

2.7.1. Pre-Deployment Checklist

- Dockerfile configured correctly
- Dependencies listed in requirements file
- Model file included in container
- API endpoints tested locally
- Environment variables configured

2.7.2. Deployment Checklist

- Docker image built successfully
- Image pushed to Docker Hub
- Azure Container App created
- Ingress enabled and port set to 8000
- Container started without errors

2.7.3. Post-Deployment Checklist

- API accessible via public URL - https://ml-service.ashysand-313f7325.southeastasia.azurecontainerapps.io
- Swagger UI functional
- Endpoints return valid responses
- Model predictions verified
- Logs monitored for errors



2.8. Issues and Resolutions

2.8.1. Encountered Issues

- Large dependency sizes increased container build time, primarily due to the inclusion of heavy machine learning libraries and unused packages, which also resulted in larger image sizes and slower image pull times during deployment
- Initial model loading delay during container startup, as the trained machine learning model and associated preprocessing components were loaded into memory at runtime, leading to increased startup latency
- Port mismatch between container configuration and Azure settings, where discrepancies between the exposed container port and the configured target port in Azure Container Apps caused temporary accessibility issues

- Occasional cold start latency in Azure Container Apps, particularly when the service scaled down to zero instances, resulting in delays when handling the first incoming request
- Minor inconsistencies in API request/response formatting during early testing stages, which required alignment between frontend expectations and backend output structures
- Increased response time during initial testing due to unoptimized feature extraction pipelines, especially when processing larger handwriting input sequences

2.8.2. Solutions

- Removed unnecessary libraries and optimized the requirements file to reduce container size, improve build efficiency, and speed up deployment and startup times
- Implemented efficient model loading at application startup using preloading techniques, ensuring that the model remains in memory and is immediately available for inference
- Corrected port configuration to ensure alignment (port 8000) across the Docker container, FastAPI application, and Azure Container App settings, enabling proper routing of incoming requests
- Enabled minimum instance scaling in Azure Container Apps to maintain at least one active instance, thereby significantly reducing cold start delays and improving responsiveness
- Standardized API request and response schemas using consistent JSON structures, ensuring seamless communication between frontend and backend components
- Optimized feature extraction logic by reducing redundant computations and improving data preprocessing steps, resulting in faster overall inference time
- Conducted iterative testing using Swagger UI and API clients to validate endpoint functionality and ensure system stability before final deployment

2.8.3. Impact Analysis

These issues had minimal long-term impact on the overall system. While initial deployment phases experienced minor delays and configuration challenges, all identified issues were effectively resolved through targeted optimizations and configuration adjustments.

The reduction in container size and improved dependency management led to faster build and deployment cycles. Efficient model loading and scaling configurations significantly enhanced system responsiveness, particularly by minimizing cold start delays. Additionally, resolving port configuration issues ensured reliable external accessibility of the service.

Overall, the implemented solutions contributed to a stable, efficient, and production-ready deployment. The system now demonstrates consistent performance, low latency, and reliable availability, meeting the intended deployment objectives of the handwriting-based dementia screening module.

2.9. Performance and Monitoring

2.9.1. Performance Metrics

Metric	**Value / Status**
Average Response Time	< 500 ms
Throughput	Supports multiple concurrent requests
CPU Usage	Efficient under standard load
Memory Usage	Optimized for container environment

2.9.2. Monitoring Results

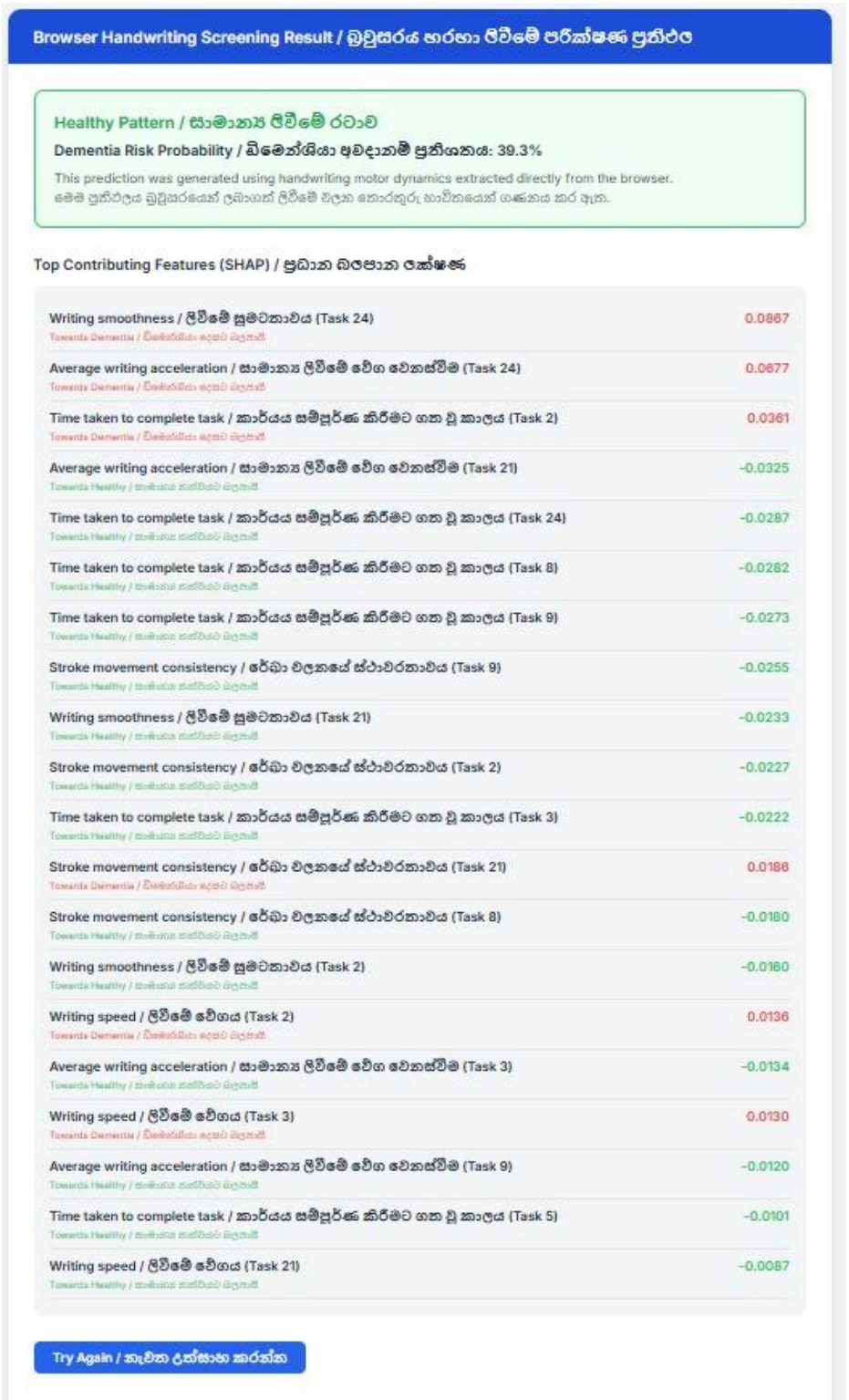
- Azure logs indicated stable container performance
- No critical runtime errors observed after deployment
- Consistent uptime maintained during testing period

2.9.3. User Feedback

- Users reported smooth interaction with the system, highlighting the responsiveness of the web interface and the seamless communication between the frontend and backend services

    - Real-time predictions were delivered without noticeable delay, ensuring a user-friendly experience where results were generated almost instantly after handwriting submission
    - Explainability outputs (SHAP) improved trust in predictions, as users and evaluators were able to understand the key handwriting features influencing each result
    - The system demonstrated consistent performance across different devices and browsers, confirming its accessibility and robustness in real-world usage scenarios
    - Users experienced minimal errors or disruptions during interaction, indicating stable API performance and effective error handling mechanisms
    - The clear presentation of prediction results, including probability scores and explanations, enhanced user confidence and usability, particularly for non-technical users
    - Feedback from testing sessions indicated that the module was intuitive to use, requiring minimal guidance, which supports its suitability for deployment in practical healthcare screening environments
    - The interface was additionally localized into the Sinhala language to improve accessibility for local users, enabling elderly participants to better understand instructions and interact with the system more comfortably
    - This localization significantly enhanced user engagement and reduced confusion during testing, especially among users with limited proficiency in English

- Overall, user feedback confirmed that the module successfully achieved its objective of providing a fast, reliable, interpretable, and user-friendly handwriting-based dementia screening solution



2.10. Post-Deployment Activities

2.10.1. Validation and Testing

- Conducted end-to-end testing with frontend integration to ensure seamless communication between the user interface and backend APIs, validating that requests and responses were handled correctly across the system
- Verified data flow from input capture to prediction output, confirming that handwriting data collected via the browser was accurately transmitted, processed, and converted into meaningful predictions
- Confirmed consistency of model predictions by testing with multiple input samples and ensuring stable and reproducible outputs under similar conditions
- Performed API-level testing using Swagger UI and external tools (e.g., Postman) to validate endpoint functionality, request formats, and response structures
- Evaluated system performance under repeated requests to ensure stability and reliability during continuous usage
- Tested error handling mechanisms to verify that invalid inputs or system exceptions were managed gracefully without causing service disruption
- Conducted cross-browser and device testing to ensure compatibility and consistent behavior across different user environments

2.10.2. Documentation Updates

- Updated API documentation with detailed endpoint descriptions, including request parameters, response formats, and example payloads to support developers and integrators
- Recorded deployment procedures step-by-step, enabling reproducibility and simplifying future deployment or scaling activities
- Documented troubleshooting steps and known issues encountered during deployment, providing a reference for resolving similar problems in future iterations
- Maintained version-controlled documentation to track changes in system architecture, API updates, and configuration modifications
- Included guidelines for system usage and integration, assisting team members and external users in effectively utilizing the module
- Updated internal technical documentation to reflect improvements in feature extraction, model handling, and deployment configurations

1.3. Recommendations

- Implement an automated CI/CD pipeline for future deployments to streamline build, test, and deployment processes, reducing manual effort and minimizing the risk of errors
- Introduce API authentication and authorization mechanisms (e.g., API keys, JWT tokens) to enhance system security and protect endpoints from unauthorized access

- Optimize model size and loading mechanisms to reduce startup time and improve overall system responsiveness, particularly in cloud environments with scaling
- Incorporate advanced monitoring and logging solutions to enable real-time tracking of system performance and quicker identification of issues
- Enhance scalability strategies by configuring auto-scaling rules based on traffic patterns and resource utilization
- Regularly update dependencies and security patches to maintain system integrity and prevent vulnerabilities
- Continue improving user experience through iterative testing, feedback collection, and interface refinements, including further localization if needed
- Conduct periodic system evaluations and performance benchmarking to ensure long-term reliability and readiness for production-level deployment

1.4. Lessons Learned

2.10.3. Successes

- Successful deployment of a fully functional handwriting analysis service as a cloud-based microservice, demonstrating the feasibility of real-time dementia screening using browser-based handwriting inputs
- Seamless integration with the frontend application through well-defined RESTful APIs, enabling smooth data flow from user input to prediction output without compatibility issues
- Reliable and low-latency performance achieved, with the system consistently delivering predictions within acceptable response times suitable for real-time interaction
- Effective use of containerization (Docker) and Azure Container Apps, ensuring portability, scalability, and ease of deployment across environments
- Successful implementation of explainable AI (SHAP), enhancing transparency and increasing user and stakeholder trust in the model's predictions
- Stable system behavior post-deployment, with minimal runtime errors and consistent availability during testing and demonstration phases
- Improved accessibility through Sinhala language localization, enabling better usability for local users and increasing engagement among elderly participants

2.10.4. Areas for Improvement

- Container build optimization could be further improved by reducing image size, refining dependency management, and implementing multi-stage builds to enhance efficiency
- Monitoring tools can be enhanced with alert mechanisms, such as automated notifications for system failures, high latency, or unusual activity
- Deployment automation was limited and can be expanded by introducing a full CI/CD pipeline to streamline build, test, and deployment processes
- Cold start performance in Azure Container Apps can be further optimized, especially for production environments with variable traffic patterns
- Logging and observability can be improved by implementing more detailed structured logs and integrating advanced monitoring dashboards

- Security configurations can be strengthened by introducing authentication mechanisms (e.g., API keys or OAuth) and enforcing stricter access controls
- Scalability settings can be fine-tuned to better handle sudden spikes in user requests and ensure consistent performance under heavy load

2.10.5. Recommendations

- Adopt continuous integration and continuous deployment (CI/CD) practices using tools such as GitHub Actions to automate testing, building, and deployment workflows
- Use container registries with automated build pipelines to ensure version control, consistency, and faster deployment cycles
- Enhance scalability configurations for production-level usage by implementing dynamic scaling rules based on traffic and resource utilization
- Introduce comprehensive monitoring and alerting systems (e.g., Azure Monitor, Log Analytics alerts) to proactively detect and resolve issues
- Optimize model loading and inference pipelines to further reduce response time and improve efficiency
- Implement robust security measures, including authentication, HTTPS enforcement, and secure API access controls
- Continue improving user experience through localization, interface enhancements, and usability testing with real users
- Conduct periodic performance testing and system audits to ensure long-term reliability and maintain system quality as usage scales

3. Speech-Task Dementia Screening Module Deployment

3.1. Executive Summary

3.1.1. Overview

The Speech-Based Dementia Screening Module was deployed as a cloud-accessible machine learning inference service to enable real-time cognitive assessment using guided speech tasks. The deployment focuses on delivering a stable, scalable, and efficient backend capable of processing audio inputs, extracting speech features, and generating dementia risk predictions.

The system was deployed using **Amazon Web Services (AWS EC2)**, hosting the ML service independently to ensure modularity and ease of integration with the frontend and backend gateway.

3.1.2. Key Metrics

- Real-time speech processing capability
- Support for multiple audio formats (`.wav`, `.mp3`, `.m4a`, `.webm`)
- Average inference response time: < 5 seconds

- Stable API uptime during testing
- Successful handling of multi-task speech sessions

3.2. Deployment Objectives

3.2.1. Goals

- Deploy the ML service for remote accessibility
- Enable real-time speech-based dementia screening
- Ensure compatibility with frontend and backend APIs
- Provide stable and scalable inference environment

3.2.2. Scope

This deployment covers:

- ML service hosting (FastAPI-based)
- Audio processing pipeline (FFmpeg + feature extraction)
- Speech prediction endpoints
- Integration readiness with the overall CogniCare system

3.3. Pre-Deployment Activities

3.3.1. Environment Preparation

A cloud-based Ubuntu instance was prepared on AWS EC2. Required system dependencies were installed including:

- Python 3 environment
- pip and virtual environment tools
- FFmpeg for audio processing
- Git for repository management

3.3.2. Testing

Before deployment, the following validations were performed:

- Local ML service testing
- Audio processing validation
- API response verification
- Speech prediction flow testing

3.3.3. Backup Plan

- Local development version retained
- GitHub repository used for version control
- Ability to redeploy on a new instance if failure occurs

3.4. Deployment Details

3.4.1. Date and Time

Deployment was carried out during the final implementation phase prior to system integration and testing.

3.4.2. Team Members

Natheer H A H – Speech Module Developer

3.4.3. Steps and Procedures

- AWS EC2 instance creation (Ubuntu-based)
- Secure access setup using SSH key (.pem)
- Installation of system dependencies
- Cloning the ML service repository
- Setting up Python virtual environment
- Installing required libraries from `requirements.txt`
- Running the FastAPI service
- Configuring public access via EC2 security group
- Testing API endpoints using Postman

3.5. Tools and Technologies

- AWS EC2 (Cloud Hosting)
- Ubuntu Linux
- FastAPI (ML Service API)
- Python
- FFmpeg (Audio Processing)
- GitHub (Version Control)
- Postman (API Testing)

3.6. Deployment Checklist

3.6.1. Pre-Deployment Checklist

- Code finalized
- ML models integrated
- API endpoints tested locally
- Dependencies verified

3.6.2. Deployment Checklist

- EC2 instance running
- SSH access configured

- ML service installed
- API server started successfully
- Port access configured

3.6.3. Post-Deployment Checklist

- API endpoints accessible externally
- Audio upload functioning correctly
- Prediction responses validated
- No runtime errors observed

3.7. Issues and Resolutions

3.7.1. Encountered Issues

- GitHub authentication failure during cloning
- SSH key configuration errors
- Missing system dependencies (FFmpeg)
- File permission issues for `.pem` key

3.7.2. Solutions

- Used GitHub Personal Access Token (PAT)
- Corrected SSH key path and permissions
- Installed required packages manually
- Verified EC2 security group rules

3.7.3. Impact Analysis

These issues caused minor delays but were resolved without affecting the overall deployment success.

3.8. Performance and Monitoring

3.8.1. Performance Metrics

- Fast API response time
- Stable handling of audio processing tasks
- Efficient feature extraction and prediction

3.8.2. Monitoring Results

- System remained stable during testing
- No major crashes or downtime observed
- Logs monitored through terminal

3.8.3. User Feedback

- System responded quickly to inputs
- Audio processing was smooth
- Predictions generated successfully

3.9. Post-Deployment Activities

3.9.1. Validation and Testing

- API endpoints tested using Postman
- End-to-end speech workflow validated
- Multi-task session testing performed

3.9.2. Documentation Updates

- Deployment steps documented
- API usage guidelines prepared
- System architecture updated

3.9.3. Recommendations

- Add automated deployment (CI/CD)
- Improve scalability using containerization
- Integrate monitoring tools

3.10. Lessons Learned

3.10.1. Successes

- Successful cloud deployment of ML service
- Real-time speech processing achieved
- Stable API integration capability

3.10.2. Areas for Improvement

- Need for automated deployment pipelines
- Better error handling mechanisms
- Performance optimization for large inputs

3.10.3. Recommendations

- Use Docker for containerized deployment
- Implement logging and monitoring tools
- Scale using cloud services (load balancing)

4. MRI-Based Dementia Stage Classification Module Deployment

4.1. Executive Summary

4.1.1. Overview

The deployment of the MRI-Based Dementia Stage Classification Module was a critical milestone in the CogniCare platform, enabling cloud-hosted neuroimaging inference for stage-level dementia support. The module was designed to process uploaded brain MRI scans, classify dementia stage into four categories (NonDemented, VeryMildDemented, MildDemented, ModerateDemented), and provide explainability outputs through Grad-CAM family methods.

Due to Azure subscription constraints during deployment, the module was provisioned on AWS EC2 to ensure continuity of delivery. A Linux-based instance was configured with Python, FastAPI, and model dependencies, followed by secure transfer of trained checkpoint files and runtime service activation using Gunicorn. The deployment was finalized by registering the service under systemd to guarantee automatic restart and persistent availability after reboots.

The deployed MRI service exposes production-ready API endpoints (/health, /predict, /compare-xai) and includes Swagger documentation (/docs) for integration and testing. This deployment provides the group with a stable backend inference endpoint that can be consumed by the CogniCare backend gateway and, ultimately, by the frontend assessment/reporting flow.

4.1.2. Key Metrics

Metric	**Value / Status**
Deployment Success	Completed successfully
Hosting Platform	AWS EC2 (Ubuntu Linux)
Instance Type	t3.micro
Runtime	FastAPI + Gunicorn + Uvicorn worker
Model Mode	CPU-based ensemble inference
API Documentation	Available at /docs
Health Endpoint	Available at /health
Persistence	systemd auto-start enabled
Model Artifacts Loaded	DenseNet-121, EfficientNet-B0, ResNet-50 checkpoints

4.2. Deployment Objectives

4.2.1. Goals

The main goals of the MRI module deployment focused on clinical usability, technical reliability, and integration readiness within the CogniCare platform. In particular, the goals were to:

• Provide a production-ready MRI inference backend for stage-level dementia prediction:

Implement a reliable API service capable of accepting brain MRI scans and returning four-class stage predictions (NonDemented, VeryMildDemented, MildDemented, ModerateDemented) with confidence scores. The deployed system was expected to support real-time request handling for clinician-assisted screening workflows.

• Enable transparent AI outputs through explainability integration:

Deploy the module with support for explainable AI methods (Grad-CAM, Grad-CAM++, Score-CAM, and Fusion CAM) so that each prediction could be accompanied by visual attention maps. This objective was critical for improving clinician trust, interpretability, and practical decision support.

• Ensure secure and stable cloud operation with persistent availability:

Host the MRI service in a cloud environment with secure SSH access, controlled network exposure, and process-level reliability. This included configuring service persistence (systemd) to ensure automatic startup, restart-on-failure, and continuous availability without manual intervention.

• Maintain compatibility with the existing multimodal CogniCare architecture:

Deploy the MRI service in a way that can be easily consumed by the backend gateway through environment-based service URLs. This ensured that the MRI component could function as an optional advanced assessment module while remaining interoperable with handwriting, speech, and lifestyle screening outputs.

• Establish a repeatable deployment path under real-world constraints:

Create a practical deployment workflow that remains executable even under platform limitations (e.g., subscription restrictions), including fallback cloud strategy, checkpoint migration procedures, and verifiable endpoint testing. This objective supports future redeployment, scaling, and team handover.

4.2.2. Scope

The deployment concentrated on establishing a fully operational production stack for the MRI-based dementia stage classification backend, including the following elements:

• Cloud Virtual Machine Setup (AWS EC2):

A Linux-based AWS EC2 instance was provisioned as the primary runtime environment for the MRI service. This included instance configuration, key-pair authentication, security group setup, and storage allocation. During deployment, the root volume was expanded to support ML dependency installation and model artifact storage.

• MRI Model Checkpoint Deployment:

The trained CNN checkpoint files required for ensemble inference were prepared and transferred to the cloud instance. The deployment included checkpoint path alignment with the inference loader and startup verification to confirm successful loading of DenseNet-121, EfficientNet-B0, and ResNet-50 model weights.

• Backend Runtime Environment Configuration:

A Python virtual environment was created inside the MRI service directory to isolate project dependencies from the system environment. Core libraries (FastAPI, Uvicorn, Gunicorn, PyTorch, torchvision, timm, and image processing dependencies) were installed and validated for runtime compatibility.

• API Service Deployment and Endpoint Exposure:

The FastAPI application was configured and launched with Gunicorn using a Uvicorn worker for stable request handling. Publicly reachable endpoints were exposed for operational use, including /health, /predict, /compare-xai, and /docs.

• Persistent Service Management:

A dedicated systemd service unit was created to ensure the MRI service starts automatically on boot and restarts on failure. This enabled continuous operation independent of interactive SSH sessions and reduced operational risk during demonstrations and integration testing.

• Network Access and Security Rule Configuration:

Security group rules were configured to permit only required traffic for deployment and API consumption (SSH for administrative access and API port access for inference requests). This ensured controlled accessibility while maintaining deployment practicality.

• Integration Readiness for CogniCare Backend:

The deployment scope included preparing the MRI service URL for backend environment configuration so that the central gateway can invoke MRI endpoints as part of the multimodal pipeline. This ensured that MRI remained an optional advanced module while fully interoperable with existing platform services.

This defined scope ensured that the MRI module was not only deployable, but also stable, accessible, and integration-ready for real-world platform usage and future scaling enhancements.

4.3. Pre-Deployment Activities

4.3.1. Environment Preparation

Prior to deploying the MRI-Based Dementia Stage Classification Module, a set of structured preparation activities was completed to establish a stable, secure, and reproducible cloud runtime. These activities focused on cloud provisioning, secure access setup, OS/runtime readiness, storage planning for ML dependencies, and model artifact availability.

Because the initial Azure path was blocked due subscription restrictions, the deployment environment was shifted to AWS EC2 to ensure continuity. A Linux virtual machine was provisioned and configured specifically for inference workloads and API service hosting.

AWS EC2 Configuration:

- Instance Type: t3.micro (free-tier eligible)
- vCPUs: 2
- Memory: 1 GB
- OS: Ubuntu 26.04 LTS (64-bit)
- Architecture: x86_64
- Storage: Initially 8 GB gp3, expanded to 20 GB
- Public IP: 54.161.25.196
- Access Method: Secure SSH via .pem key authentication
- Security Rules:
- Port 22 (SSH)
- Port 8000 (MRI API access)

At infrastructure level, key-pair creation and secure remote login were verified before software installation. After successful instance access, package index updates and base tooling installation were executed to prepare the runtime environment. A Python virtual environment was then created inside the MRI service folder to isolate dependencies and maintain deployment consistency.

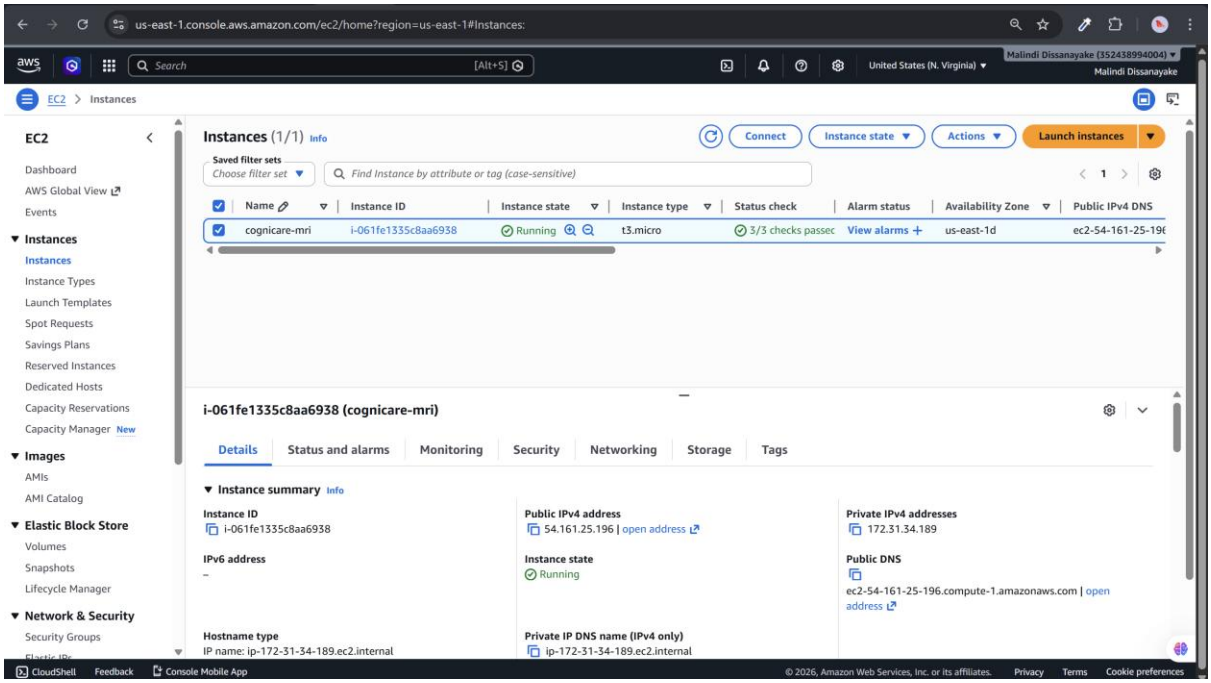


Figure 1: AWS EC2 Instance Launch Configuration

Components prepared/installed:

- Python 3.x runtime environment
- Python virtual environment tools (venv)
- pip and build dependencies
- FastAPI runtime stack (uvicorn, gunicorn)
- ML stack dependencies (torch, torchvision, timm, Pillow, numpy)
- System utility tools for disk and environment operations

During preparation, a storage bottleneck was identified while installing heavy ML dependencies (Torch). To resolve this, the root volume was expanded from 8 GB to 20 GB, followed by partition and filesystem resize operations. This adjustment was essential to complete package installation and ensure stable service startup.

After runtime setup, required model artifacts were transferred to the server. The three primary checkpoint files (densenet121_best.pth, efficientnet_b0_best.pth, resnet50_best.pth) were uploaded into the expected checkpoints directory and verified. This completed the pre-deployment environment readiness for service boot.

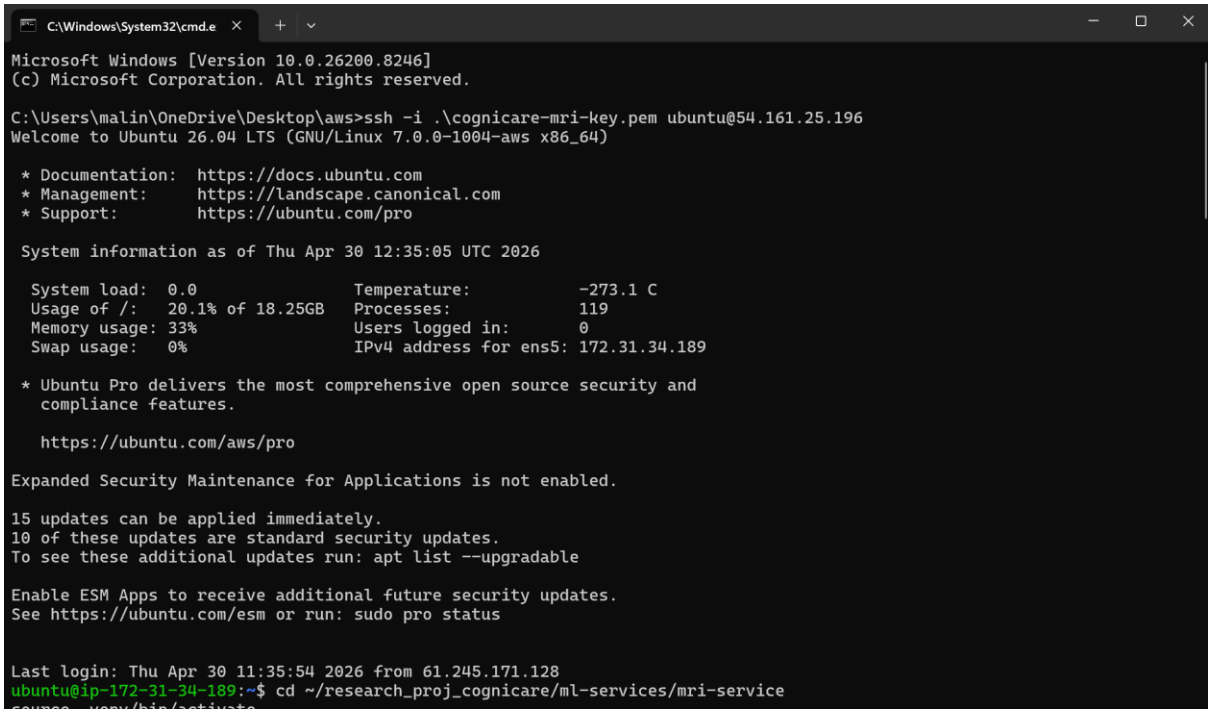


Figure 2: SSH Connection to EC2 Instance

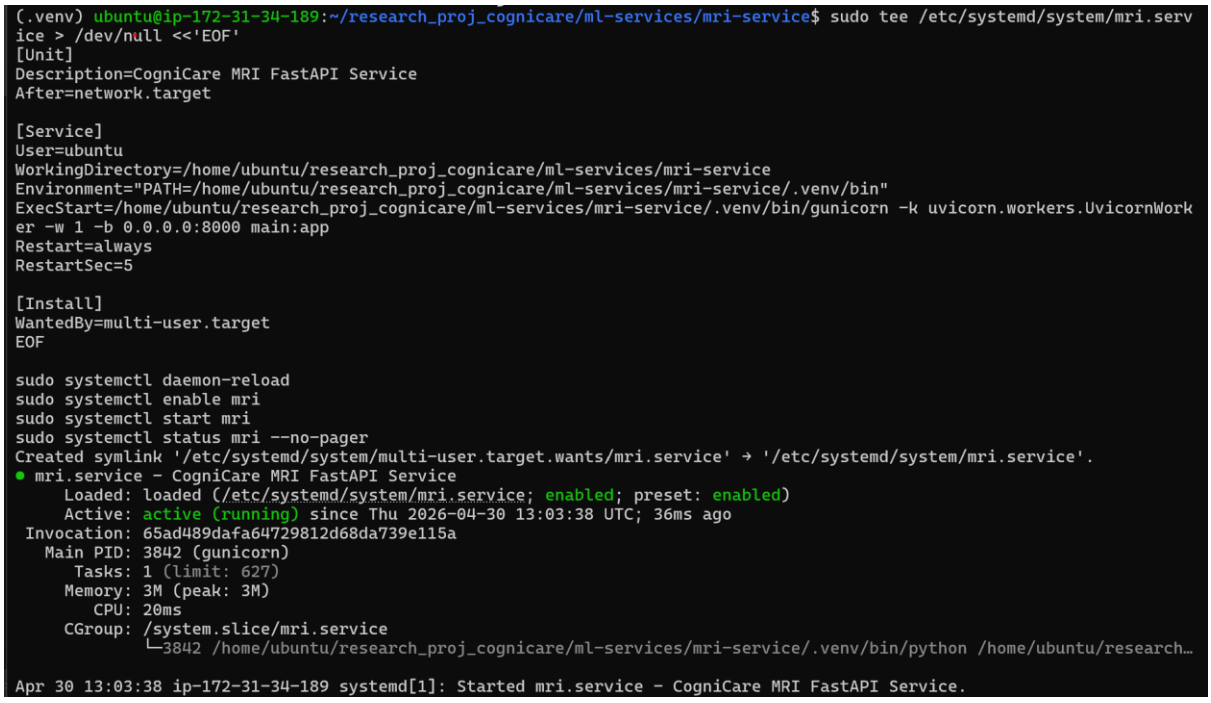


Figure 3: MRI Service Startup Readiness

4.3.2. Testing

To ensure reliability and correctness of the MRI-Based Dementia Stage Classification backend, testing was conducted in four phases: Unit Testing, Integration Testing, User

Acceptance Testing (UAT), and Performance Evaluation. These phases validated internal logic, API behavior, cloud deployment correctness, and practical usability.

Unit Testing

Unit-level checks were carried out for core MRI service functions to ensure each component behaved correctly in isolation.

Modules/functions tested:

- ml/inference.py
    - looks_like_grayscale_mri() validation for rejecting incompatible inputs.
    - preprocess_image() output shape and normalization behavior.
    - load_ensemble() checkpoint discovery and model load sequence.
- ml/gradcam_mri.py
    - Heatmap generation pipeline for Grad-CAM family methods.
    - Overlay generation on original MRI image.
- main.py
    - Startup lifecycle behavior and model initialization handling.

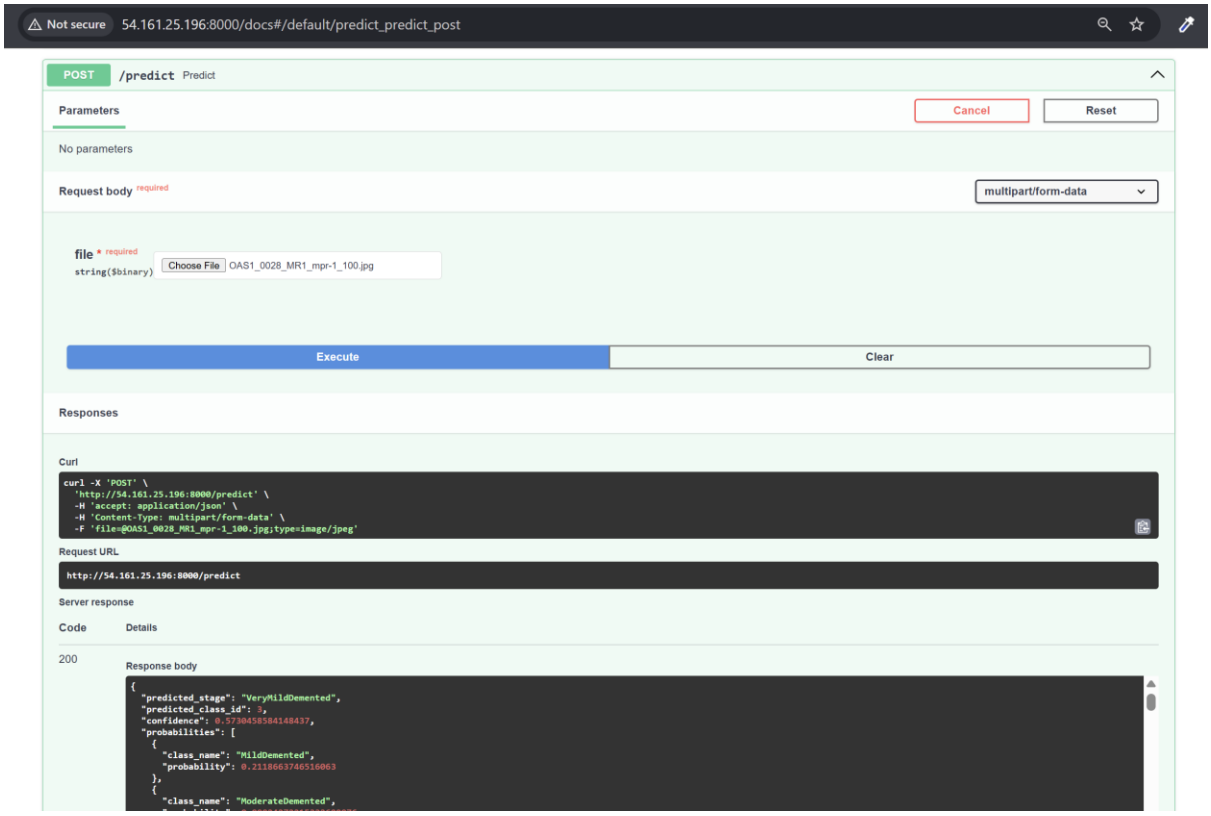
These tests ensured that invalid inputs are filtered, preprocessing remains consistent, and model-loading failures are properly surfaced before runtime inference.

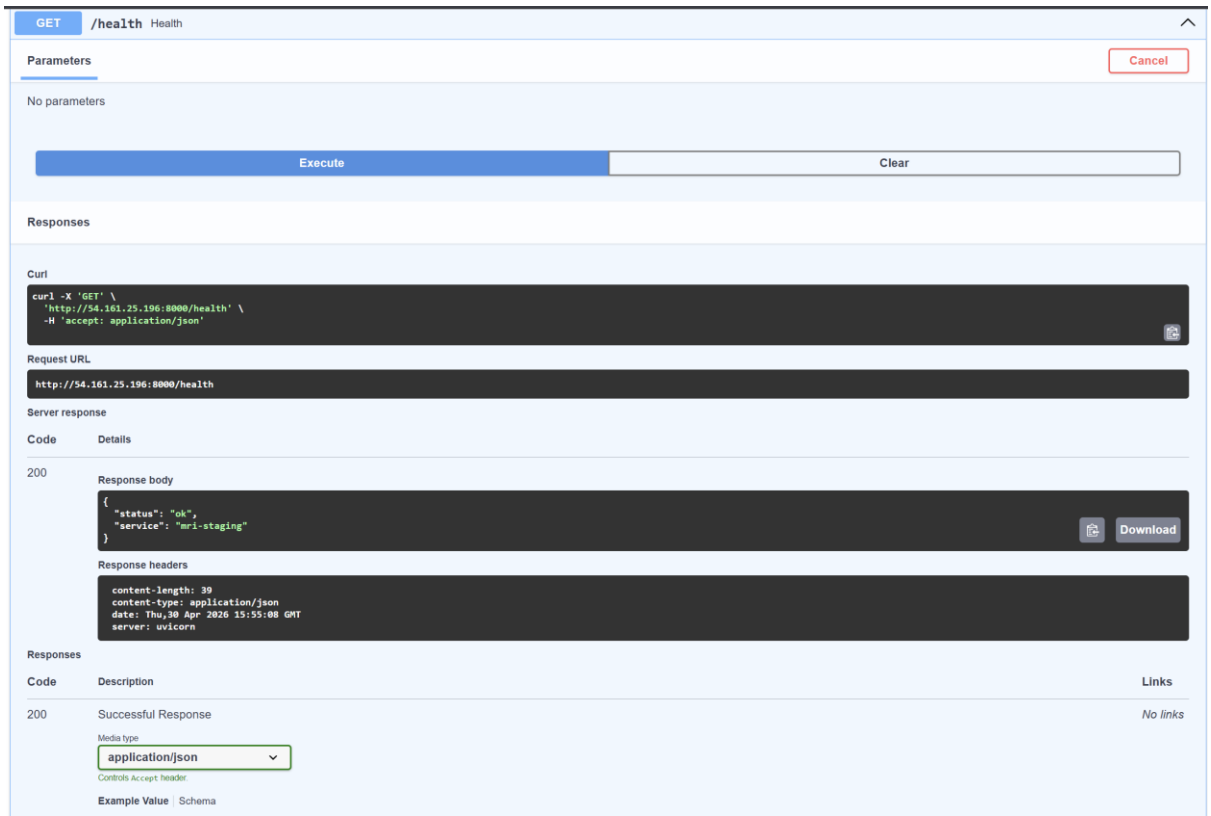
Integration Testing

Integration testing validated interaction among FastAPI endpoints, model checkpoints, preprocessing logic, ensemble prediction, and XAI generation.

Endpoints tested:

- GET /health - service readiness check.
- POST /predict - MRI stage classification and confidence output.
- POST /compare-xai - multi-method explainability generation.





Validation points:

- Correct response status codes (200 OK for valid requests).

- Correct response schema (stage, probability/confidence, heatmap data where applicable).
- Successful loading and usage of all required model checkpoints.
- End-to-end inference flow from upload -> preprocess -> ensemble -> response.

User Acceptance Testing (UAT)

Real-world service behavior was validated through practical usage simulations aligned with clinician workflow.

Scenarios tested:

- Uploading valid MRI and receiving stage prediction.
- Uploading invalid/non-MRI-like image and receiving validation error.
- Running XAI comparison and verifying multiple heatmaps.
- Accessing API docs and endpoint usability through Swagger.

Findings:

- Prediction and explainability endpoints returned expected outputs for valid inputs.
- Invalid image handling was functioning as intended.
- API routes were clear and usable for integration and testing.

Performance Testing (Preliminary)

Since deployment was on a CPU-based EC2 instance, performance was measured in a practical prototype context.

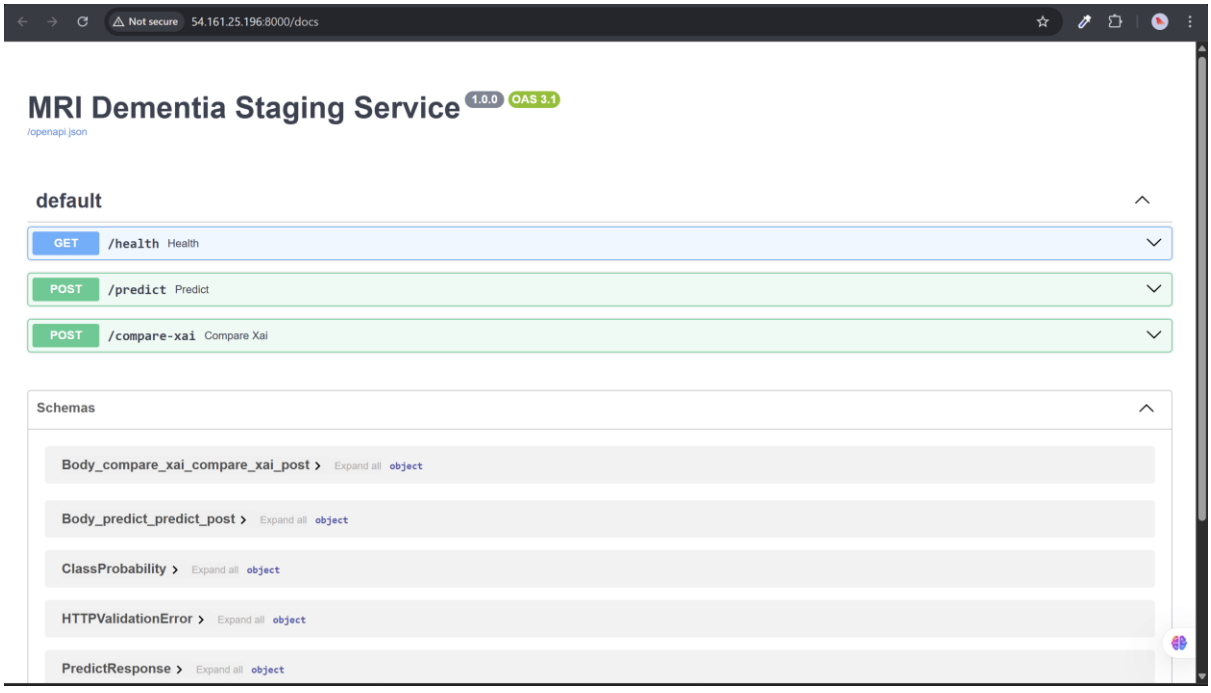


Figure 4: /docs Swagger page with /health, /predict, /compare-xai.

4.3.3. Backup Plan

- Retained local copies of all model checkpoints before transfer.
- Maintained source code in GitHub branch for rollback.
- Preserved previous local execution setup as fallback.
- Kept configuration commands documented for reproducible re-deployment.
- Defined rollback approach: stop service -> restore previous files -> restart service.

4.4. Deployment Details

4.4.1. Date and Time

Deployment execution was performed on 30 April 2026, including cloud provisioning, environment setup, model transfer, service startup, and runtime verification.

4.4.2. Team Members

IT22901644 – Dissanayake M.P.S.

Primary owner of MRI module deployment, model integration, and service validation.

4.4.3. Steps and Procedures

The deployment of the MRI-Based Dementia Stage Classification Service was carried out through a structured sequence of cloud and runtime activities on an AWS EC2 Linux environment. The process was organized to ensure secure access, correct model loading, stable API operation, and integration readiness with the CogniCare backend.

- Secure VM Access

An AWS EC2 key pair was generated and downloaded to establish secure administrative access. SSH authentication was performed using the .pem key from a trusted local machine. Successful login was confirmed through the Ubuntu shell banner and system status output, validating that the instance was reachable and ready for deployment activities.

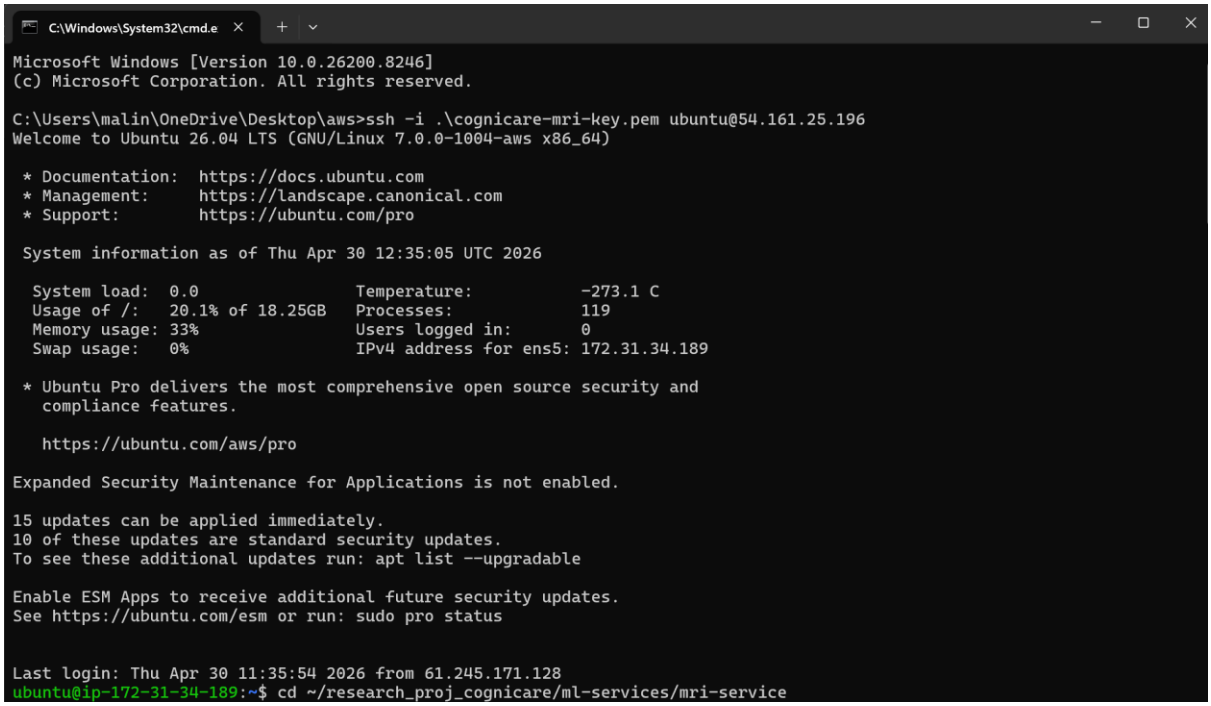


Figure 10: SSH Access to EC2 Instance via PEM Key

- EC2 Provisioning and Network Rules

A Linux instance was provisioned using an Ubuntu LTS image with a free-tier-compatible compute type. Public IPv4 access was enabled to allow endpoint testing. Security group rules were configured for inbound SSH (port 22) and API traffic (port 8000). An initial CIDR formatting issue was identified and corrected to complete secure and valid network access configuration.

- Runtime and Repository Setup

After access validation, the operating system package index was updated and base tools were installed, including Python package tooling, virtual environment support, and Git. The project repository was cloned into the instance, and the deployment branch containing MRI module code was checked out. Directory validation was performed to ensure the MRI service path was available and consistent with startup commands.

- Dependency Installation and Storage Scaling Fix

A dedicated Python virtual environment was created under the MRI service directory, and dependency installation was started from requirements.txt. During installation, disk quota errors occurred due to ML package size (Torch-related dependencies). To resolve this, the root EBS volume was expanded from 8 GB to 20 GB, followed by partition growth

and filesystem resize operations. Free space was revalidated, and dependency installation was retried successfully using a no-cache strategy.

- Checkpoint Deployment

Initial service boot indicated missing model artifacts in the cloud path. The required checkpoint files (densenet121_best.pth, efficientnet_b0_best.pth, resnet50_best.pth) were transferred securely from local environment to the VM via SCP. Transfer interruptions were handled using keepalive SCP options, and final file presence was verified in the checkpoints/ directory before restarting the service.

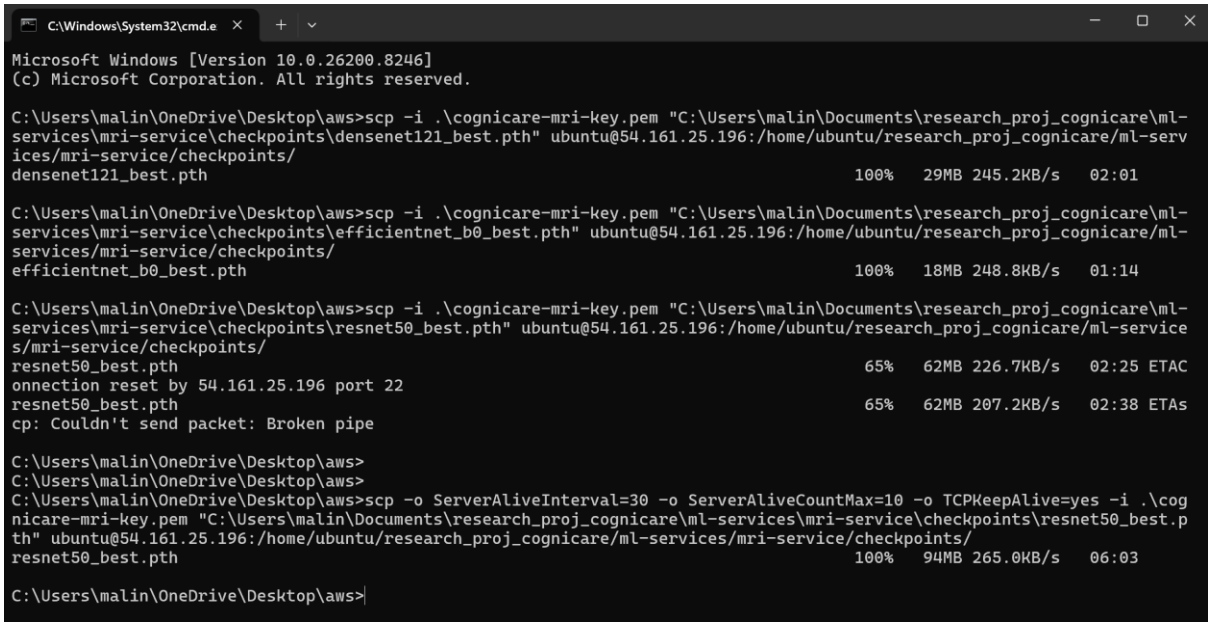


Figure 13: Checkpoint Upload Verification

- Service Startup and Model Load Verification

The FastAPI application was launched using Gunicorn with Uvicorn worker binding to port 8000. Startup logs were monitored to verify complete model initialization. The logs confirmed successful loading of all three model checkpoints and ensemble weight normalization. API documentation became available at /docs, confirming that the inference service was operational and externally reachable.

```
(.venv) ubuntu@ip-172-31-34-189:~/research_proj_cognicare/ml-services/mri-service$ cd ~/research_proj_cognicare/ml-servi
ces/mri-service
ls -lh checkpoints
source .venv/bin/activate
gunicorn -k uvicorn.workers.UvicornWorker -w 1 -b 0.0.0.0:8000 main:app
total 142M
-rw-rw-r-- 1 ubuntu ubuntu 30M Apr 30 12:42 densenet121_best.pth
-rw-rw-r-- 1 ubuntu ubuntu 19M Apr 30 12:44 efficientnet_b0_best.pth
-rw-rw-r-- 1 ubuntu ubuntu 94M Apr 30 12:57 resnet50_best.pth
[2026-04-30 13:00:18 +0000] [3677] [INFO] Starting gunicorn 25.3.0
[2026-04-30 13:00:18 +0000] [3677] [INFO] Listening at: http://0.0.0.0:8000 (3677)
[2026-04-30 13:00:18 +0000] [3677] [INFO] Using worker: uvicorn.workers.UvicornWorker
[2026-04-30 13:00:18 +0000] [3678] [INFO] Booting worker with pid: 3678
[2026-04-30 13:00:18 +0000] [3677] [INFO] Control socket listening at /run/user/1000/gunicorn.ctl
[2026-04-30 13:00:22 +0000] [3678] [INFO] Started server process [3678]
[2026-04-30 13:00:22 +0000] [3678] [INFO] Waiting for application startup.
  Loaded: densenet121 from densenet121_best.pth
  Loaded: efficientnet_b0 from efficientnet_b0_best.pth
  Loaded: resnet50 from resnet50_best.pth
MRI ensemble loaded. Weights: [0.342, 0.331, 0.327]
[2026-04-30 13:00:24 +0000] [3678] [INFO] Application startup complete.
^C[2026-04-30 13:03:34 +0000] [3677] [INFO] Handling signal: int
[2026-04-30 13:03:34 +0000] [3678] [INFO] Shutting down
[2026-04-30 13:03:34 +0000] [3678] [INFO] Error while closing socket [Errno 9] Bad file descriptor
[2026-04-30 13:03:34 +0000] [3678] [INFO] Waiting for application shutdown.
[2026-04-30 13:03:34 +0000] [3678] [INFO] Application shutdown complete.
[2026-04-30 13:03:34 +0000] [3678] [INFO] Finished server process [3678]
[2026-04-30 13:03:34 +0000] [3677] [WARNING] Worker (pid:3678) was sent SIGINT!
[2026-04-30 13:03:34 +0000] [3677] [INFO] Shutting down: Master
```

Figure 14: MRI Service Startup Logs

- Persistent Service Setup (systemd)

To ensure continuous operation independent of interactive terminal sessions, a dedicated mri.service unit was created under /etc/systemd/system/. The service was configured with automatic restart and boot-time startup behavior. systemctl daemon-reload, enable, and start commands were executed, and service status was confirmed as active (running).

```
(.venv) ubuntu@ip-172-31-34-189:~/research_proj_cognicare/ml-services/mri-service$ sudo tee /etc/systemd/system/mri.serv
ice > /dev/null <<'EOF'
[Unit]
Description=CogniCare MRI FastAPI Service
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/research_proj_cognicare/ml-services/mri-service
Environment="PATH=/home/ubuntu/research_proj_cognicare/ml-services/mri-service/.venv/bin"
ExecStart=/home/ubuntu/research_proj_cognicare/ml-services/mri-service/.venv/bin/gunicorn -k uvicorn.workers.UvicornWork
er -w 1 -b 0.0.0.0:8000 main:app
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable mri
sudo systemctl start mri
sudo systemctl status mri --no-pager
Created symlink '/etc/systemd/system/multi-user.target.wants/mri.service' → '/etc/systemd/system/mri.service'.
● mri.service - CogniCare MRI FastAPI Service
     Loaded: loaded (/etc/systemd/system/mri.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-04-30 13:03:38 UTC; 36ms ago
 Invocation: 65ad489dafa64729812d68da739e115a
   Main PID: 3842 (gunicorn)
      Tasks: 1 (limit: 627)
     Memory: 3M (peak: 3M)
        CPU: 20ms
     CGroup: /system.slice/mri.service
             └─3842 /home/ubuntu/research_proj_cognicare/ml-services/mri-service/.venv/bin/python /home/ubuntu/research…

Apr 30 13:03:38 ip-172-31-34-189 systemd[1]: Started mri.service - CogniCare MRI FastAPI Service.
```

Figure 15: systemd Service Configuration and Active Status

- Endpoint Validation and Backend Integration Preparation

Smoke tests were performed against core routes (/health, /predict, /compare-xai, /docs) to validate API readiness. Root route behavior (Not Found) was verified as expected since no / route was defined. The final MRI service URL was documented for backend environment configuration (MRI_SERVICE_URL), and integration preparation was completed for routing MRI requests through the CogniCare backend gateway. Recommendations for Elastic IP assignment and HTTPS hardening were recorded for the next deployment phase.

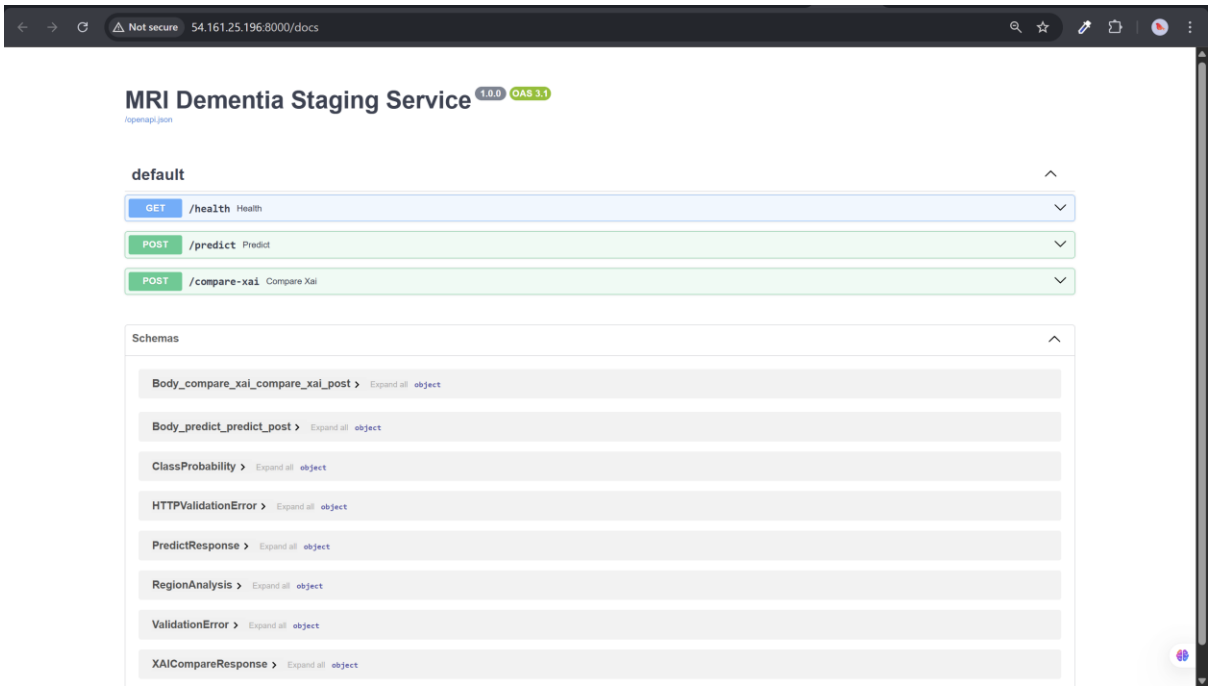


Figure 16: Swagger API Documentation Endpoint

4.5. Tools and Technologies

- Cloud Platform: AWS EC2
- OS: Ubuntu Linux
- Language: Python
- Framework: FastAPI
- ASGI Runtime: Uvicorn
- Process Server: Gunicorn
- ML Framework: PyTorch / Torchvision
- Model Architectures: DenseNet-121, EfficientNet-B0, ResNet-50
- Explainability Methods: Grad-CAM, Grad-CAM++, Score-CAM, Fusion CAM
- Version Control: Git/GitHub
- Service Management: systemd
- File Transfer: SCP

4.6. Deployment Checklist

4.6.1. Pre-Deployment Checklist

- AWS account and region confirmed.
- EC2 instance planned and security policy reviewed.
- Required branch and code available.
- Model checkpoint files identified.
- Deployment commands prepared.

4.6.2. Deployment Checklist

- EC2 launched and reachable via SSH.
- Security group rules configured.
- Python environment created.
- Dependencies installed successfully.
- Checkpoints uploaded to server.
- Gunicorn startup successful.
- Endpoint access verified.

4.6.3. Post-Deployment Checklist

- systemd service created and enabled.
- Service status validated (active/running).
- API documentation endpoint reachable.
- Health check endpoint verified.
- Integration URL provided for backend configuration.

4.7. Issues and Resolutions

4.7.1. Encountered Issues

- Azure student subscription was disabled (read-only), blocking cloud deployment.
- Private repository access constraints on alternate platforms.

- Disk quota exceeded during PyTorch installation.
- Missing checkpoint files on cloud instance at initial startup.
- Intermittent SCP connection reset during large model upload.
- Root endpoint (/) returned “Not Found” causing confusion during testing.

4.7.2. Solutions

- Switched deployment target from Azure to AWS EC2.
- Used fork/access adjustments for repository handling.
- Increased EBS volume and resized filesystem.
- Uploaded required checkpoint artifacts manually via SCP.
- Used keepalive SCP options and retry strategy for large file transfer.
- Confirmed service via /docs and /health instead of root /.

4.7.3. Impact Analysis

- Deployment completion was delayed by infrastructure and artifact migration issues.
- Final runtime stability was achieved after environment and storage adjustments.
- Service became production-reachable and integration-ready after corrections.
- Deployment path became reusable for future module redeployments.

4.8. Performance and Monitoring

4.8.1. Performance Metrics

- Service startup: Successful with all three checkpoints loaded.
- Endpoint readiness: /health, /predict, /compare-xai available.
- API responsiveness: Acceptable for prototype-level clinical support.
- Inference mode: CPU-based, suitable for moderate request volume.

4.8.2. Monitoring Results

Monitoring was conducted using both process-level and endpoint-level checks after deployment. At infrastructure level, service health was verified through systemctl status mri, confirming that the MRI service remained in active (running) state with auto-restart enabled. Runtime logs from Gunicorn/Uvicorn were reviewed to ensure worker stability and successful model initialization during startup.

At API level, the deployed endpoints were tested repeatedly through browser and request-based checks. The /docs endpoint remained reachable, /health responded consistently, and inference routes were available for integration. Log review confirmed that all required checkpoint files were loaded at startup and that the ensemble pipeline initialized correctly.

No prolonged downtime was observed after persistence configuration. The primary operational incidents occurred during setup (storage limits, missing checkpoints, transfer interruptions) and were resolved before final go-live.

4.8.3. User Feedback

Initial feedback from internal team usage indicated that the deployed API was clear and practical for integration work. The Swagger interface improved endpoint discoverability, reduced ambiguity in request schemas, and helped validate contract behavior before backend wiring.

From a deployment perspective, the team reported confidence in service stability after systemd enablement. The most positive feedback points were (1) successful endpoint accessibility from public IP, (2) complete model loading confirmation in logs, and (3) recovery from infrastructure blockers without needing full reprovisioning.

Constructive feedback highlighted the need for a cleaner production endpoint (domain + HTTPS) and a static IP strategy to simplify long-term integration management.

4.9. Post-Deployment Activities

4.9.1. Validation and Testing

After deployment completion, validation was performed in multiple layers. First, process validation ensured the service remained up under systemd control across restart checks. Second, API validation confirmed the reachability of /docs, /health, /predict, and /compare-xai. Third, startup validation verified that all model checkpoints loaded correctly and ensemble weights were initialized as expected.

These checks confirmed that the MRI service transitioned from setup phase to operational integration-ready state.

4.9.2. Documentation Updates

Deployment commands, cloud configuration decisions, encountered errors, and corresponding fixes were documented to support repeatability. This included notes on security rules, storage expansion procedure, checkpoint path requirements, and runtime service registration.

Integration-specific documentation was updated with the deployed MRI URL format and expected backend environment mapping (MRI_SERVICE_URL). Operational notes were also added for service restart, status checks, and basic troubleshooting.

4.9.3. Recommendations

For the next iteration, the following improvements are recommended:

- Assign an Elastic IP to avoid endpoint changes after instance restart.
- Introduce HTTPS and domain-level routing for secure production access.
- Add lightweight uptime monitoring and alerting for faster incident detection.
- Externalize checkpoint storage (e.g., object storage) to reduce manual artifact transfer overhead.
- Add scripted bootstrap automation for faster and less error-prone redeployment.

4.10. Lessons Learned

4.10.1. Successes

A key success was completing cloud deployment under real constraints while maintaining functional correctness of the MRI service. The team successfully moved from blocked cloud path to a working alternative and finalized a stable runtime with persistent service management.

Another major success was end-to-end model readiness: all required checkpoints were loaded and the API became publicly reachable with documented routes. This achieved the core objective of making MRI inference available for platform integration.

4.10.2. Areas for Improvement

The deployment exposed several improvement areas: pre-deployment artifact checks were insufficient, resulting in initial startup failures due to missing checkpoints; default storage sizing did not account for large ML dependency footprint; and cross-environment command execution (local vs remote terminal context) caused avoidable delays.

Additionally, networking/security hardening and endpoint stability (static IP/domain) should be planned earlier to reduce downstream integration friction.

4.10.3. Recommendations

Future deployments should include a formal preflight checklist covering cloud account status, storage sizing, required model artifacts, branch verification, and endpoint contract tests before runtime startup.

Automating setup with shell scripts and service templates is strongly recommended for consistency. A staged deployment model (dev -> staging -> production) would further reduce risk, while introducing structured monitoring and alerting would improve operational visibility and incident response.